Lightweight Object Detection for African Penguin Population Monitoring

Submitted in fulfilment

of the requirements for the degree of

BACHELOR OF SCIENCE (HONOURS)

of Rhodes University

Hluma Xako

Grahamstown, South Africa

October 15, 2026

Declaration of Authorship

I, Hluma Xako, declare that this thesis titled, *Lightweight Object Detection for African Penguin Population Monitoring* and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I consulted the published work of others, this is always clearly attributed.
- Where I quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.
- AI was used to help revise and refine the thesis, including improvements to academic writing, structure and flow, and debugging code. It was also used to assist with checking the consistency of numerical results and citations against results and sources.

Signed: HX

Date: 16/10/2026

Abstract

The African Penguin (*Spheniscus demersus*) has declined by approximately 73% in South Africa over the past three decades and is now listed as Critically Endangered. Detecting further decline depends on accurate and repeatable population counts, but the surveys currently relied upon are labour-intensive and difficult to sustain at isolated colonies. Automated computer vision offers a scalable alternative, but penguin detectors have mostly been evaluated on dedicated graphics processing units (GPUs), leaving their performance on the standard computers available to field teams unknown.

This study tested whether lightweight YOLO detectors can count penguins accurately on a central processing unit (CPU) alone. A dataset of 37,487 images from citizen-science, fixed-camera and public datasets was labelled with the Segment Anything Model 3, verified by hand, and split 70/20/10 with related photographs kept together. The nano and medium versions of YOLOv11 and YOLOv26 were evaluated on detection accuracy, counting accuracy and CPU latency, using Intel OpenVINO and native PyTorch on an Intel Core Ultra 5 235.

The nano models remained within 4.6 points of the medium models on mean average precision (mAP@50). OpenVINO was faster than PyTorch for every model, by 1.34 to 1.56 times rather than the hypothesised two times, and the nano models processed 63 to 68 photographs per second. The medium models counted the test set to within about 1% of the true total but were seven times slower. Between the nano models, detection accuracy was almost identical and YOLOv26n was only 7% faster than YOLOv11n, well below the 43% reported by its developers. YOLOv26n, however, undercounted the test set by 11.1%, against 5.4% for YOLOv11n, which came within 1.9% on African Penguins. As the count is what monitoring needs, YOLOv11n with OpenVINO is recommended for counting African Penguins on standard CPU hardware.

ACM Computing Classification System

Thesis classification under the ACM Computing Classification System[1]:

- **Computing methodology ∼ Neural Networks**
- **Computing methodology ∼ Object Detection**
- *Applied computing ∼ Computer Vision for Ecological Monitoring*

General-Terms: African Penguin, Object Detection, YOLO, Wildlife Monitoring, Lightweight Deep Learning, CPU Inference.

[1] https://www.acm.org/publications/class-2012

Acknowledgements

The authors would like to thank the authors of the publicly available datasets. This work was undertaken in the Distributed Multimedia CoE at Rhodes University, with financial support from Telkom SA. The authors acknowledge that opinions, findings and conclusions or recommendations expressed here are those of the author(s) and that the above-mentioned sponsor accepts no liability whatsoever in this regard.

I would also like to thank everyone who supported me, directly and indirectly throughout this project. In particular, I am deeply grateful to my supervisor, Professor Dane Brown, for his continued guidance and encouragement throughout the year. His detailed feedback on every draft, his high standards and the time he gave to this project, including curating several of the datasets it relies on, have shaped both this thesis and the way I approach research. I could not have completed this work without his mentorship.

I would also like to thank the Rhodes University Department of Computer Science, the lecturers, who pour so much of their time and energy to teaching.

Finally, thank you to my family and friends for their unwavering support and encouragement throughout this year.

Contents

1 Introduction **1**

1.1 Background 1

1.2 Context of Research 2

1.3 Problem Statement 3

1.4 Research Objectives 4

1.5 Approach 4

1.6 Thesis Outline 5

2 Literature Review **6**

2.1 Wildlife Monitoring and Population Estimation 6

2.1.1 Penguin-Specific Detection and Monitoring Strategies 7

2.1.2 Density-Based and Hybrid Counting 9

2.1.3 Open-World Counting Models 10

2.1.4 Camera Trap-Based Wildlife Detection 11

2.2 Deep Learning Architectures for Object Detection 12

2.2.1 Convolutional Neural Networks 12

2.2.2 Two-Stage vs. One-Stage Object Detection 14

2.2.3 YOLO-Based Detection Architectures 15

2.2.4 YOLOv11 17

2.2.5 YOLOv26 18

2.3 Hardware-efficient Inference and Model Deployment 19

2.4 Datasets and Annotation Strategies 21

2.5 Critical Analysis of Literature 22

3 Methodology and Implementation 24

3.1 Methodology Overview 24

3.2 Dataset Construction 25

3.2.1 Data Sources 26

3.2.2 Annotation Pipeline 26

3.2.3 Merging and Deduplication 27

3.2.4 Data Split 28

3.3 Model Selection and Training 29

3.4 Hyperparameter Tuning and Scale Augmentation 31

3.5 Deployment and Benchmarking 32

3.6 Experimental Setup 33

3.7 Summary 35

4 Results and Discussion **36**

4.1 Experiment 1: Detection and Counting Accuracy 36

4.1.1 Experiment 1.1: Original Split 36

4.1.2 Experiment 1.2: Grouped Split 36

4.2 Experiment 2: CPU Inference Latency 39

4.3 Experiment 3: Numerical Precision 41

4.4 Experiment 4: Hyperparameter Tuning 42

4.5 Experiment 5: Scale Augmentation 43

4.6 Accuracy-Efficiency Trade-off 44

4.7 Summary 45

5 Conclusion and Future Work **47**

5.1 Conclusion 47

5.2 Contributions 48

5.3 Limitations 49

5.4 Future Work 49

References **50**

Appendices **56**

A Appendix **56**

A.1 Training Curves 56

A.2 Counting by Number of Penguins per Photograph 56

A.3 Example Detections 59

A.4 Full FP32 and FP16 Results 59

A.5 Latency Across Benchmarking Sessions 59

A.6 Hyperparameter Tuning Trials 60

A.7 Scale Augmentation on African Penguins 61

List of Figures

2.1 Overview of single-stage object detection architectures, demonstrating the unified pipeline in which bounding box localisation and class prediction are performed simultaneously within a single network (Ali and Zhang, 2024). . 15

3.1 Overview of the model development methodology. 25

4.1 Detection accuracy against mean CPU latency per image on the grouped-split test set. Filled points use OpenVINO FP32 and hollow points native PyTorch, with accuracy measured using OpenVINO. The latency axis is logarithmic. 44

A.1 Training on the original split (Experiment 1.1): training box loss (a) and validation mAP@50:95 (b) per epoch. 57

A.2 Training on the grouped split (Experiment 1.2): training box loss (a) and validation mAP@50:95 (b) per epoch. 57

A.3 Counting error of each model by the number of penguins per test photograph, on the grouped split at a confidence threshold of 0.25. The shaded band in (b) marks the 5% target. 58

A.4 YOLOv11n detections on four African Penguin test photographs from iNaturalist, at a confidence threshold of 0.25. Each title gives the true and predicted counts. 58

A.5 The ten tuning trials of Experiment 4.2. (a) Validation mAP@50:95 against learning rate, with a fitted trend. (b, c) The score remaining after the learning-rate trend is removed, against momentum and weight decay. ρ is Spearman's rank correlation. 60

List of Tables

3.1 Images contributed by each source after duplicate removal (37,487 in total). GEORa was used for training only, and all other sources were divided between training, validation and test sets (Section 3.2.4). 27

3.2 Training, validation and test sets for the two splits. Excluding the 3,635 GEORa training images, the grouped split is exactly 70/20/10. 29

3.3 Hardware and software used for training and benchmarking. 31

4.1 Experiment 1.1: detection and counting accuracy on the original split (855 African Penguin test photographs). Detection values were measured with the OpenVINO FP32 models on the CPU. These models were trained with the optimiser chosen automatically by Ultralytics and, for YOLOv11m, a batch size of 8 (Section 3.3), and were tested on a different test set, so their results are not directly comparable with Table 4.2. 37

4.2 Experiment 1.2: detection accuracy on the grouped split, measured with the OpenVINO FP32 models on the CPU. Precision and recall are given at the confidence that maximises F1. 37

4.3 Counting accuracy by the number of penguins per test photograph, on the grouped split at a confidence threshold of 0.25. Total error is the difference between the predicted and true totals within each group, with negative values showing undercounts. The same results are shown as a figure in Appendix A.2. 39

4.4 Experiment 1.2: counting accuracy on the grouped split at a confidence threshold of 0.25. MAE, within 5% and total error use the photographs containing penguins: 2,825 in the full test set and all 835 African Penguin photographs. Total error is the difference between the predicted and true totals, with negative values showing undercounts. False boxes are detections on the 558 test photographs without penguins, none of which are African Penguin photographs. 39

4.5 Experiment 2: mean CPU latency per image (± standard deviation) at FP32 on the grouped-split test set. Speed-up is PyTorch latency divided by OpenVINO latency. 40

4.6 Experiment 3: OpenVINO models at FP32 and FP16 precision on the grouped-split test set. Latency is the mean per image. 41

4.7 Experiment 4: hyperparameter tuning of YOLOv11n. The tuned model on the grouped split was evaluated by the tuning script with PyTorch. 42

4.8 Experiment 5: YOLOv11n trained with two scale settings, evaluated on the full grouped-split test set. Detection was measured with OpenVINO FP32 on the CPU, and counting at a confidence threshold of 0.25. 43

A.1 OpenVINO models at FP32 and FP16 on the grouped-split test set, timed in the second benchmarking session. “All” is the full test set and “African” its 835 African Penguin photographs. 59

A.2 OpenVINO FP32 latency per image (± standard deviation) in the two benchmarking runs, on the same 3,383 test photographs. Difference is the change from the Experiment 2 run to the Experiment 3 run, as a percentage of the Experiment 2 run. 59

A.3 The ten tuning trials of YOLOv11n on the grouped split (Experiment 4.2), ordered by learning rate. Each trial continued the baseline for 20 epochs. The baseline scored 0.6651. 60

A.4 Experiment 5 detection results for YOLOv11n on the full test set and on its African Penguin photographs. 61

Glossary

The acronyms used in this thesis are listed below.

AP	Average Precision
C2PSA	Cross-Stage Partial with Spatial Attention
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CPU	Central Processing Unit
CSP	Cross Stage Partial
CUDA	Compute Unified Device Architecture
DETR	Detection Transformer
DFL	Distribution Focal Loss
dHash	Difference Hash
FP16	16-bit Floating Point
FP32	32-bit Floating Point
FPN	Feature Pyramid Network
FPS	Frames Per Second
GFLOPs	Billions of Floating-Point Operations
GPU	Graphics Processing Unit
iGPU	Integrated Graphics Processing Unit
INT8	8-bit Integer
IoU	Intersection over Union
IR	Intermediate Representation
IUCN	International Union for Conservation of Nature
LVIS	Large Vocabulary Instance Segmentation
MAE	Mean Absolute Error
mAP	mean Average Precision
MD5	Message-Digest Algorithm 5
MuSGD	Muon and SGD hybrid optimiser
NMS	Non-Maximum Suppression
ONNX	Open Neural Network Exchange
OpenVINO	Open Visual Inference and Neural Network Optimization
R-CNN	Region-based Convolutional Neural Network
ResNet	Residual Network
RPN	Region Proposal Network
RT-DETR	Real-Time Detection Transformer
SA-Co	Segment Anything with Concepts
SAM	Segment Anything Model
SD	Standard Deviation
SGD	Stochastic Gradient Descent
SPPF	Spatial Pyramid Pooling-Fast
SSD	Single Shot MultiBox Detector
STAL	Small-Target-Aware Label Assignment
UAV	Unmanned Aerial Vehicle
VGG	Visual Geometry Group
YOLO	You Only Look Once

1

Introduction

1.1 Background

The African Penguin (*Spheniscus demersus*) is an endemic seabird species distributed along the coastlines of Namibia and South Africa. Characterised by its black-and-white plumage and speckled stomach, this species plays an integral ecological role in the health of marine ecosystems and is considered an indicator of the availability of small pelagic fish (BirdLife International, 2024). Despite this importance, the population in South Africa has declined by approximately 73% over the past three decades (Sherley *et al.*, 2021, cited in Van den Berg 2025), and the species is now listed as Critically Endangered on the IUCN Red List (BirdLife International, 2024).

This decline is largely attributed to the cumulative effects of historical exploitation, such as egg harvesting and guano scraping, together with current pressures from commercial fishing and climate change (Geldenhuys, 2018). Commercial fishing has reduced sardine and anchovy stocks off Namibia and western South Africa, while climate change has shifted the remaining fish southward, away from many breeding colonies (Geldenhuys, 2018). African Penguins are also sensitive to temperature, which limits where they can breed. In the same study, sea-surface temperature was the largest contributor to habitat suitability, accounting for 72.4% in the annual model, followed by mean land temperature (Geldenhuys, 2018).

Detecting whether this decline continues, and whether conservation measures are effective, requires accurate data on population change (Sherley *et al.*, 2010). Sherley *et al.* (2010) note that traditional methods, such as marking individual birds with flipper bands, provide sparse data at a high cost in time and effort, and that flipper bands have been linked to higher swimming costs and lower survival in penguins. They also report that only about 13% of birds at Robben Island carried a flipper band in 2007.

Automated counting from photographs avoids handling the birds and could reduce this effort, but it must work under two constraints. First, penguins in field photographs are often small, overlap one another and blend into rock and guano, especially in dense colonies. Second, the computer available in the field is assumed to be a standard computer with a CPU and no dedicated GPU, similar to the consumer laptop used by Sherley *et al.* (2010) for their field system on Robben Island. This thesis tests whether a lightweight object detector can count African Penguins accurately under both constraints.

1.2 Context of Research

African Penguin conservation depends on accurate and scalable population censuses to detect declines early and evaluate conservation efforts. In this thesis, a census refers to a complete count of the penguins in the surveyed images rather than an estimate from a sample, and census-level accuracy refers to a total count within 5% of the true total. Historically, manual monitoring methods involved ground surveys to count nests or chicks, a process that is time-consuming, labour-intensive, and prone to human error during the incubation season (Liu *et al.*, 2020). Two experts counting the same colony images, for example, disagreed by up to 11.2% (Liu *et al.*, 2020). Monitoring is further complicated by the geographical isolation of colonies, which are often located on remote islands and coastlines. While automated computer vision offers a solution, current high-performance models typically necessitate dedicated GPU hardware (Tuia *et al.*, 2022). This creates a technical barrier for field researchers who rely on standard computers or edge devices with limited computational power. Lightweight detection models that maintain high accuracy

while remaining deployable on CPU-based hardware would make automated monitoring accessible in the field, ideally in real time. In this thesis, real-time means processing at least 25 images per second, the frame rate of standard video, on a standard CPU.

Several studies have explored automated wildlife detection and monitoring using computer vision techniques. These approaches include aerial imagery analysis, satellite remote sensing and camera trap imagery. For penguins, Liu *et al.* (2020) counted nests in drone images with Faster R-CNN, coming within 2% of expert counts on half of their orthomosaics, while Wu *et al.* (2023) detected Adélie penguins with a YOLO-based model that ran at 43 frames per second, more than twice as fast as Faster R-CNN. Recent research has increasingly focused on applying object detection models, especially YOLO-based architectures, due to their speed and the range of model sizes available, from nano to extra-large.

1.3 Problem Statement

Existing penguin detection systems demonstrate high mAP and near-expert counts, but rely on dedicated GPU hardware and high-resolution imagery (Liu *et al.*, 2020, Wu *et al.*, 2023). However, the trade-off between computational efficiency and census-level counting accuracy remains insufficiently explored. In particular, the deployment feasibility of these methods under CPU-only field conditions has not been adequately addressed. This research therefore evaluates whether lightweight object detectors can detect and count penguins on a CPU, staying within 5 points of larger models on mAP@50 and counting the total number of penguins within 5% of the true total. This target is stricter than the 11.2% disagreement between two expert counters reported by Liu *et al.* (2020). The share of individual photographs whose own count falls within 5% of their true count is also reported, but the target applies to the total.

1.4 Research Objectives

Based on the identified research gap, this study is guided by the following research objectives:

1. Determine whether lightweight YOLO models can detect and count African Penguins in field photographs within 5 points of larger models on mAP@50 and within 5% of the true total count.
2. Determine whether OpenVINO reduces the CPU latency of these models compared with native PyTorch, and how many images per second they can process on a standard CPU.
3. Quantify how much detection and counting accuracy is lost for the speed gained when moving from medium to nano YOLO models, and whether this gap can be narrowed without reducing speed.

It is hypothesised that a lightweight YOLO-based model, optimised for CPU inference via OpenVINO export, can achieve a mAP@50 within 5 points of a higher-capacity baseline and a total count within 5% of the true total, while reducing mean inference latency by at least 2× on CPU hardware. Throughout this thesis, differences in mAP are given in points, where one point is a difference of 0.01.

1.5 Approach

This section summarises the approach, which is described in full in Chapter 3. Images from several public sources were merged into one single-class dataset of 37,487 images (Section 3.2.1). The primary source was African Penguin (*Spheniscus demersus*) photographs from iNaturalist (iNaturalist, 2026), supplemented with photographs of three closely related species, colony images from the GEORa camera of the Penguin Watch project (Jones *et al.*, 2018), and seven colony and aerial datasets hosted on Roboflow

Universe. All images were labelled with SAM 3 (Carion *et al.*, 2026), and every box was then checked and corrected by hand (Section 3.2.2). Duplicate images were removed with a perceptual hash, and the remaining images were divided 70/20/10 into training, validation and test sets, keeping photographs from the same iNaturalist observation, and copies of the same image, on the same side of the split (Sections 3.2.3 and 3.2.4). An earlier split, which evaluated African Penguin photographs only, is also reported for comparison.

Four models were trained under identical settings: the nano and medium versions of YOLOv11, the conventional YOLO design, and of YOLOv26, which removes Distribution Focal Loss and offers an NMS-free detection head (Jocher *et al.*, 2026) (Section 3.3). Each model was exported to the Intel OpenVINO toolkit at FP32 and FP16 precision and benchmarked against native PyTorch on an Intel Core Ultra 5 235 CPU, without a GPU (Section 3.5). Two further experiments tested whether the accuracy of YOLOv11n could be raised without changing its speed, through hyperparameter tuning and a narrower scale augmentation (Section 3.4).

The models were evaluated on detection accuracy, using mAP@50 and mAP@50:95, on counting accuracy, using the mean absolute error per photograph and the error in the total count, and on CPU latency, using the mean time per image and frames per second (Section 3.6).

1.6 Thesis Outline

Chapter 2 reviews the literature on automated wildlife monitoring, object detection architectures, CPU deployment, and datasets and annotation. Chapter 3 describes how the dataset was built and how the models were trained and benchmarked. Chapter 4 presents and discusses the results of the five experiments, and Chapter 5 concludes the thesis and outlines future work.

2

Literature Review

This chapter reviews the literature on automated wildlife monitoring, with a focus on penguin detection and population counting using computer vision. It examines how counting methods and detection architectures have developed, as well as the approaches used to deploy these models on hardware with limited computational resources. A recurring theme in the literature is that most proposed systems achieve strong detection accuracy or counting performance when evaluated on dedicated GPU hardware while their suitability for CPU-based field deployment is frequently unexplored.

This chapter is organised as follows. Section 2.1 discusses approaches to wildlife population monitoring, Section 2.2 covers deep learning architectures for object detection, Section 2.3 reviews methods for efficient inference on CPU hardware, Section 2.4 examines datasets and annotation strategies, and Section 2.5 summarises the gaps that motivate this study.

2.1 Wildlife Monitoring and Population Estimation

This section reviews existing approaches to automated wildlife population estimation. The section begins with penguin-specific detection and monitoring strategies, then covers density-based counting methods and hybrid methods that combine density estimation with object detection. It then examines open-world counting models, which can count new object types without retraining but rely on large foundation models, and ends with

camera trap studies that share many of the visual challenges of colony monitoring. Across these studies, inference speed and CPU-based deployment are rarely reported, which is the gap this study addresses.

2.1.1 Penguin-Specific Detection and Monitoring Strategies

Edney and Wood (2021) reviewed digital imaging platforms for seabird monitoring including satellite imagery, aerial surveys, and fixed-position time-lapse camera systems. The review found that each platform involves trade-offs between resolution, cost, and operational feasibility. Time-lapse cameras were identified as a cost-effective option for long-term colony monitoring as images can be captured at set intervals with only occasional maintenance visits required. However, this approach requires the colony to remain visible from a fixed viewpoint. The review also highlights limitations in automated counting. Simple automated counters, such as ImageJ, can undercount dense colonies when overlapping birds are detected as a single object, while the Penguin Watch team's Pengbot model required approximately 82,000 volunteer annotated images for training and testing. The computing hardware required to run these automated methods in field conditions was not addressed. As a review, it reports no detection or counting accuracy of its own.

Wu *et al.* (2023) investigated automatic penguin detection in high-resolution remote sensing images of Antarctic Adélie penguin colonies, and proposed YoloPd, an object detection network designed to address the problem of small-object detection in complex backgrounds. Building on YOLOv7, YoloPd incorporated a multi-frequency feature fusion module (MAConv) to preserve fine-scale features, a bottleneck-efficient aggregation layer (BELAN) to enable deeper feature learning, and a transformer-based aggregation layer (TBELAN) to reduce background confusion from rock and guano. On sub-images, YoloPd achieved an mAP@50 of 89.4% and an mAP@50:95 of 40.9%, outperforming Faster R-CNN by 8.5 points and YOLOv7 by 1.9 points in mAP@50:95. On 18 full test images, it achieved an average counting accuracy of 94.6%. However, the additional feature-processing modules introduced a computational cost, with YoloPd achieving 43 FPS compared with 66 FPS for YOLOv7 and 20 FPS for Faster R-CNN. All models were trained on a single NVIDIA V100

GPU, and no CPU results were reported, leaving deployment feasibility under CPU-only conditions unexamined.

Liu *et al.* (2020) approached penguin colony counting in UAV imagery by stitching overlapping high-resolution frames into orthomosaics, ensuring that each nest appeared only once. A two-stage Faster R-CNN with a ResNet-101 backbone and a Feature Pyramid Network was trained separately for two nesting phases, namely penguins incubating eggs and adults guarding chicks, with anchor sizes reduced to (8, 16, 32, 64, 128) to improve small-object recall. A dark channel prior was applied to mitigate fog interference, and images containing very small penguins were upscaled until each penguin measured approximately 40×40 pixels. After comparing several architectures, the authors selected Faster R-CNN because it detected small, densely packed nests more reliably than one-stage detectors, accepting longer training and inference times in exchange. Performance was reported as nest counts rather than mAP. The automated count fell within 2% of an expert count on five of ten orthomosaics. These results should be read against the variability of manual counting itself, as two experts counting the same orthomosaics disagreed by up to 11.2%. The system ran on a workstation with two NVIDIA GPUs, a GTX 1070 and a 2080, and neither inference speed nor CPU performance was reported.

Both Wu *et al.* (2023) and Liu *et al.* (2020) demonstrate that automated detection can approach the accuracy of expert counts in penguin monitoring. YoloPd achieved an average counting accuracy of 94.6% on Adélie imagery, and the Faster R-CNN pipeline came within 2% of expert counts on five of ten UAV orthomosaics. These figures are not directly comparable, however. Wu *et al.* (2023) report average counting accuracy for individual penguins, while Liu *et al.* (2020) report the difference from expert nest counts for each orthomosaic. Both studies also used aerial imagery and GPU hardware, and neither evaluates performance under CPU-only constraints. The transferability of these results to ground-level and fixed-camera imagery processed on standard field hardware therefore remains an open question.

2.1.2 Density-Based and Hybrid Counting

Density-based counting approaches estimate object counts by predicting spatial density maps rather than explicit detections, with the total count obtained by summing the map. Arteta *et al.* (2016) applied this paradigm to penguin counting using fixed-camera imagery of wild colonies from the Penguin Watch project (Jones *et al.*, 2018), where adverse weather conditions, variation in penguin scale, visual similarity between birds and background elements such as rocks, extreme crowding and inter-occlusion made individual penguins difficult to detect. The dataset was annotated through crowdsourcing, whereby volunteers placed a dot on each penguin. This reduced annotation cost but introduced inconsistencies between annotators and missing instances, producing a noisy ground truth. Jones *et al.* (2018) quantified this noise, finding that volunteer counts differed from expert counts by 0.88 to 2.36 adult penguins per image across four cameras. To address this, Arteta *et al.* (2016) incorporated foreground-background segmentation into the density estimation pipeline, using the segmentation output to generate improved density targets. Compared with a density-only baseline, this reduced the mean counting error from 7.09 to 3.26 penguins per image when test images came from cameras seen during training, and from 5.01 to 3.41 on unseen cameras, although error grew with crowding, from 2.2 to 12.5 penguins per image between the least and most crowded images. As a density-based method, it produces no bounding boxes, so no detection metric applies, and neither the hardware used nor the inference speed was reported.

Detection-based methods such as YOLO, R-CNN and SSD perform well in sparse scenes but have yet to achieve satisfactory results in extremely dense crowds (Gao *et al.*, 2025). The shift toward density estimation addressed this, but density maps yield no explicit object localisation, precluding the bounding box outputs required for individual tracking. Phan *et al.* (2024) proposed DEO-Net to recover both, jointly performing density estimation and object detection through a dual-branch network with an attention-based fusion module. DEO-Net achieved state-of-the-art results on four crowd counting benchmarks, including an MAE of 54.2 on ShanghaiTech Part A. However, the additional branch and fusion module increase computational cost, and all training and evaluation was conducted on

NVIDIA RTX A5000 GPUs. As with Arteta *et al.* (2016), no inference latency or CPU deployment results were reported. Detection-based models, by contrast, retain explicit localisation without the need for a second branch.

2.1.3 Open-World Counting Models

Open-world counting models accept multi-modal prompts to enumerate object instances without being retrained for each new object type. Amini-Naieni *et al.* (2024) introduced CountGD, a counting model built on the GroundingDINO vision-language foundation model, which accepts text descriptions, visual exemplars, or both to specify the target object. CountGD achieved a test MAE of 5.74 on the FSC-147 benchmark when provided with both prompt types, and reports counting error rather than detection accuracy. However, two limitations constrain its applicability to ecological monitoring. First, the authors acknowledge that the model fails to distinguish between adult and chick penguins due to their visual similarity in colour and shape. Second, CountGD is a large foundation model trained on a 48 GB NVIDIA A6000 GPU, and no inference speed or CPU results were reported, so its suitability for CPU-constrained field hardware remains untested.

GroundingDINO nonetheless remains applicable in an offline annotation context. Unlike online deployment, where a model must run continuously on field hardware under strict latency and power constraints, offline annotation only requires a single pass over a fixed image set on GPU-equipped laboratory hardware. In this setting, computational cost is a practical inconvenience rather than a deployment barrier. Ren *et al.* (2024) combine GroundingDINO with SAM in Grounded SAM, which reached 48.7 mean AP on the zero-shot Segmentation in the Wild benchmark, and present automatic image annotation as a key application, noting that it reduces the cost of human labelling. This supports the use of large foundation models as a labelling tool, discussed further in Section 2.4, while lightweight detectors remain better suited to deployment.

2.1.4 Camera Trap-Based Wildlife Detection

Camera-trap imagery presents visual challenges similar to those of seabird colony monitoring, including variable lighting, background clutter and visually similar individuals. Srinithi *et al.* (2024) evaluated pretrained YOLOv8, YOLOv9, and YOLOv10 on a dataset of 16,489 camera trap images spanning seven species. YOLOv10 achieved the highest precision (0.956), recall (0.902) and F1-score (0.928), with an mAP@50 of 0.929 and an mAP@50:95 of 0.675, which the authors attribute partly to its NMS-free architecture. YOLOv9, however, reached a higher mAP@50 of 0.958, and neither the hardware used nor inference speed was reported. Conventional detectors apply NMS as a post-processing step, retaining the highest-scoring bounding box and discarding any overlapping box above a set IoU threshold. In densely packed scenes, this can suppress valid detections of individuals that genuinely overlap (Hosang *et al.*, 2017), a limitation directly relevant to crowded penguin colonies.

Schneider *et al.* (2020) investigated three factors affecting deep learning performance on camera trap imagery: training set size, generalisation to new locations, and class imbalance. Using a Parks Canada dataset of 47,279 images across 55 species from 36 locations, six CNN architectures were trained to classify whole images by species, so the study reports classification accuracy rather than detection metrics. With data augmentation and transfer learning, DenseNet201 achieved 95.6% top-1 accuracy on locations seen during training. On unseen camera locations, however, the best model reached only 68.7%, which the authors attribute to overfitting to background features rather than the target species. Species with fewer than 500 training images also produced highly variable recall (0.750 ± 0.329), while those with more than 1,000 images achieved consistent recall (0.971 ± 0.014). The models were trained on an NVIDIA P100 GPU, and inference speed was not reported. Together, these results identify annotation volume and domain diversity as key determinants of generalisation.

2.2 Deep Learning Architectures for Object Detection

The wildlife monitoring approaches reviewed above share a common technical foundation: convolutional neural networks trained to recognise visual patterns in images. However, detection accuracy and computational efficiency are frequently treated as independent concerns, with models evaluated on dedicated GPU hardware and their performance on constrained platforms left as an open question. For population monitoring, the count is the output that matters most, and an object detector provides it by counting its boxes while also locating each individual. Detection accuracy therefore matters mainly through its effect on the count, and a faster detector is only useful if its count stays accurate. This section examines the architectural choices that govern this trade-off. It first introduces convolutional networks, then compares two-stage and one-stage detectors, and finally reviews the YOLO family, focusing on YOLOv11 and YOLOv26.

2.2.1 Convolutional Neural Networks

Convolutional Neural Networks form the architectural foundation of modern object detection systems. Unlike fully connected networks, which connect every pixel to every neuron and ignore spatial structure, CNNs apply small learned filters that slide spatially across an input image. Each filter produces a feature map that records where a local pattern, such as an edge, texture or shape, appears (Bhatt *et al.*, 2021). Because the same filter is reused at every position, its number of weights does not depend on the image size: a 3×3 filter on a colour image has 27 weights, whereas connecting every pixel of a 640×640 colour image to a single neuron would require over 1.2 million. This makes CNNs well-suited to images where the same pattern can appear anywhere.

A defining property of deep CNNs is the hierarchical nature of the representations they learn. Shallow layers respond to low-level features such as edges and colour gradients, while intermediate layers compose these into parts, and deeper layers integrate them into high-level semantic concepts (Bhatt *et al.*, 2021). This hierarchical abstraction is why backbone depth matters for wildlife detection. Distinguishing a penguin from visually

similar background elements such as guano, rock or shadow requires higher-level feature discrimination learned in deeper layers.

Building deeper networks, however, introduces the degradation problem, where adding layers to a plain network causes training error to increase even in the absence of overfitting. He *et al.* (2016) addressed this by introducing residual connections, which add a block's input directly to its output so that each block only needs to learn the change to its input rather than a completely new output. This makes deep networks easier to train for two reasons. If a block is not needed, it can learn a change close to zero and pass its input through unchanged, so adding layers no longer makes the network worse. During backpropagation, the gradient also flows back through the shortcut without passing through the block's weights, so it does not shrink as it travels toward the early layers. This allowed networks of over 100 layers to be trained, and the resulting ResNet family achieved a 28% relative improvement on COCO object detection over VGG-based baselines (He *et al.*, 2016). Residual connections are now built into most detection backbones, including the CSPDarknet and C3k2 blocks used in the YOLO family (Wang *et al.*, 2020, Ali and Zhang, 2024), and Liu *et al.* (2020) chose a ResNet-101 backbone for penguin counting for this reason.

Networks trained on large-scale datasets such as ImageNet, which contains over 1.2 million images across 1,000 categories, learn visual features that transfer well to new tasks (He *et al.*, 2016). Through transfer learning, these weights serve as a strong initialisation for fine-tuning, allowing a detection model to adapt to a new domain with less labelled data than training from scratch. Detectors are commonly initialised in the same way from COCO, which contains 2.5 million labelled object instances in 328,000 images (Lin *et al.*, 2014). In ecology, labels are expensive, so pretrained weights matter: the Penguin Watch team's Pengbot model, for example, required approximately 82,000 volunteer-annotated images (Edney and Wood, 2021).

2.2.2 Two-Stage vs. One-Stage Object Detection

Object detection architectures are broadly categorised into two-stage and one-stage detectors, a distinction that carries significant implications for deployment feasibility in resource-constrained environments. Two-stage detectors follow a sequential pipeline in which a first stage generates candidate object regions and a second stage classifies and refines each proposal independently. The R-CNN family represents the canonical progression of this paradigm, culminating in Faster R-CNN (Ren *et al.*, 2015), which introduced a Region Proposal Network (RPN) that shares the backbone's feature map with the detection head, enabling both stages to be trained together. Despite this integration, the architecture remains fundamentally sequential: the RPN must complete its proposals before the second-stage classifier can operate. This makes two-stage detectors slower than one-stage alternatives (Wu *et al.*, 2023) (on the same GPU, Faster-RCNN ran at 20 FPS against 66 FPS for YOLOv7), a cost likely to be greater on CPU hardware, where the parallelism advantages of dedicated GPU infrastructure are unavailable.

One-stage detectors address this bottleneck by reframing detection as a single dense prediction problem. Rather than proposing and then classifying regions, these architectures predict bounding box coordinates and class probabilities from a shared feature map in a single forward pass as illustrated in Figure 2.1. SSD (Liu *et al.*, 2016) demonstrated that competitive accuracy was achievable without a proposal stage by predicting at multiple scales from different backbone layers. YOLO (Redmon *et al.*, 2016) pursued a simpler formulation, dividing the image into a fixed spatial grid and predicting boxes and class scores directly from each cell. By eliminating both the RPN and the second-stage classifier, one-stage detectors achieve lower inference latency, making them better suited to real-time and hardware-constrained deployment.

A further distinction within one-stage architectures concerns the treatment of bounding box priors. Earlier YOLO generations relied on predefined anchor boxes, which are fixed box shapes derived from clustering the training dataset, to constrain predictions (Ali and Zhang, 2024). Anchors improve recall by biasing predictions toward expected object shapes, but they must be tuned for each domain. Anchors derived from the varied

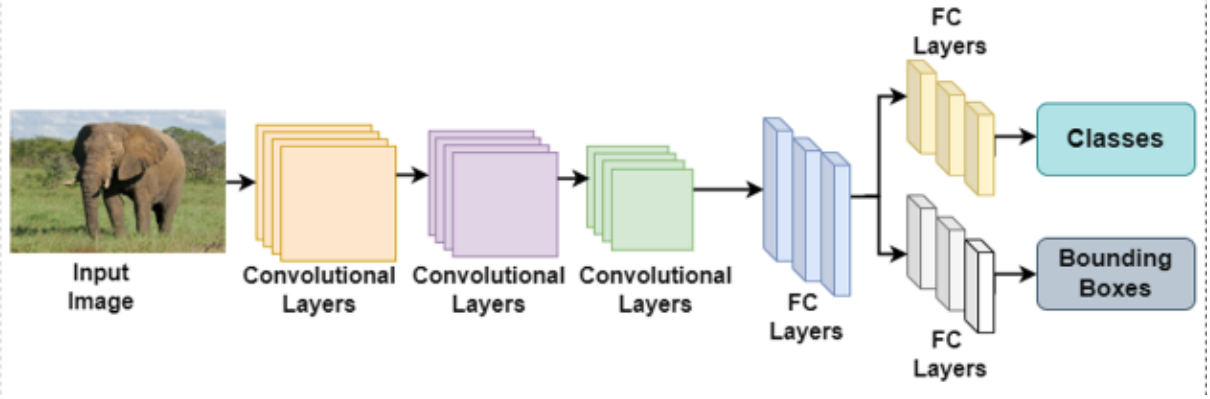


Figure 2.1: Overview of single-stage object detection architectures, demonstrating the unified pipeline in which bounding box localisation and class prediction are performed simultaneously within a single network (Ali and Zhang, 2024).

objects in COCO, for example, may be poorly suited to the compact, roughly elliptical morphology of penguins in colony imagery. Architectures from YOLOv8 onward have adopted anchor-free prediction heads that regress box dimensions directly, reducing the need to tune hyperparameters for a new domain (Ali and Zhang, 2024).

A more fundamental architectural shift has come from transformer-based detectors. DETR (Carion *et al.*, 2020) removed anchors and NMS post-processing entirely with a transformer encoder-decoder and a set-based matching loss, producing a genuinely end-to-end detection pipeline. Its successors, most notably RT-DETR (Zhao *et al.*, 2024), retain this NMS-free formulation while introducing real-time capable encoders competitive with YOLO-family models on standard benchmarks. The absence of NMS is most valuable for dense scenes, where suppression can discard valid detections of overlapping individuals. However, RT-DETR's real-time results were reported on GPU hardware, and its performance on CPUs is less established. NMS-free YOLO designs such as YOLOv10 and YOLOv26 (Jocher *et al.*, 2026) bring the same benefit to the lightweight YOLO architecture, as discussed in Section 2.2.3.

2.2.3 YOLO-Based Detection Architectures

The YOLO (You Only Look Once) family of detectors reframes object detection as a single-stage regression problem. Rather than generating region proposals and classifying them

sequentially, YOLO divides the input image into a grid of $S \times S$ cells and simultaneously predicts bounding box coordinates and class probabilities for each cell in a single forward pass (Redmon *et al.*, 2016). Modern YOLO models are organised into three functional components. A backbone CNN commonly based on CSPDarknet (Wang *et al.*, 2020) is responsible for hierarchical feature extraction. A neck then fuses multi-scale feature maps through mechanisms such as Feature Pyramid Networks (Lin *et al.*, 2017), and a detection head produces bounding box and class scores. Most YOLO versions then apply NMS to eliminate redundant predictions (Hosang *et al.*, 2017). Successive YOLO generations have introduced anchor-free prediction heads, IoU-based loss functions such as CIoU, and Distribution Focal Loss (DFL), progressively narrowing the accuracy gap with two-stage detectors while preserving single-pass inference speed (Ali and Zhang, 2024).

Persistent challenges across YOLO variants include small object detection, environmental variability, and limited computing resources. These are directly relevant to African Penguin monitoring, where individuals appear as small, visually cluttered instances against complex rock and guano backgrounds and must be detected on modest hardware. Recent work has addressed these challenges in two ways: adding modules that improve accuracy, and removing steps that cost inference time.

On the VisDrone2019 drone dataset, Wang *et al.* (2025a) replaced the neck, detection head and box loss of YOLOv11s, raising mAP@50 from 39.5% to 51.3%, above even the larger YOLOv11m (45.1%), but increasing computation from 21.3 to 36.1 GFLOPs. For ten wetland bird species, Wang *et al.* (2025b) added a module combining dilated convolutions and channel attention to the backbone of YOLOv11n, reaching an mAP@50 of 96.4%, only 0.3 points above the unmodified YOLOv11n. This model ran at 72 FPS on video using an NVIDIA V100 GPU, but the speed of the baseline was not reported. Such modules therefore add computation for gains that range from large to marginal, so their accuracy gains must be weighed against the added inference cost.

The second approach removes post-processing instead, and the NMS-free design of YOLOv10 is the clearest example. Minh *et al.* (2025) compared YOLOv10, YOLOv11 and YOLOv12 for fruit ripeness detection. At 640×640, YOLOv10n took 12.1 ms per image,

against 14.1 ms for YOLOv11n and 18.5 ms for YOLOv12n, while YOLOv12l achieved the highest accuracy, with an mAP@50:95 of 82.17%. The speed advantage did not hold at the large size, where YOLOv10l (42.2 ms) was slower than YOLOv11l (33.6 ms). All timings were measured on a Tesla T4 GPU, and the authors note that field systems typically run on low-power devices such as the Raspberry Pi, where larger models may be infeasible without optimisation such as quantisation, at some cost to accuracy.

YOLOv26 (Jocher *et al.*, 2026) takes this second direction furthest, removing DFL, making NMS optional and introducing Small-Target-Aware Label Assignment (STAL) to strengthen training on small objects. Its design and the evidence supporting it are discussed in Section 2.2.5.

Despite these advances, the majority of studies evaluate YOLO-based architectures under controlled GPU conditions and report accuracy against standard benchmarks. CPU inference latency and the trade-off between accuracy and efficiency on constrained hardware remain less studied in ecological monitoring, where object appearance, scene complexity and hardware constraints differ from standard benchmark settings (Tuia *et al.*, 2022).

2.2.4 YOLOv11

YOLOv11 extends the efficiency gains of YOLOv10 through two key architectural innovations (Ali and Zhang, 2024). The C3k2 block in the backbone improves feature aggregation while reducing computational overhead, and the C2PSA (Cross-Stage Partial with Spatial Attention) block enhances the model's ability to focus on the most informative regions of complex scenes (Ali and Zhang, 2024). The inclusion of Spatial Pyramid Pooling-Fast (SPPF) further accelerates feature aggregation and smaller kernel sizes improve processing speed without degrading detection performance. For bounding box regression, YOLOv11 employs CIoU loss, which accounts for overlap area, centre distance, and aspect ratio, with Distribution Focal Loss. Unlike YOLOv10, it still relies on NMS post-processing. YOLOv11 therefore represents the conventional YOLO design against which NMS-free and DFL-free models such as YOLOv26 can be compared (Section 2.2.5).

In benchmark evaluation, YOLOv11 achieved the highest mAP@50 of all models tested, including YOLOv10, YOLOv9, YOLOv8 and YOLO-NAS, on blood cell detection (0.958), facemask detection (0.962) and shellfish monitoring (0.563) (Ali and Zhang, 2024). The shellfish benchmark is the most relevant of these to colony monitoring. Shellfish colonies share several visual properties with penguin colonies, namely small, densely clustered instances with partial occlusion against textured backgrounds. However, shellfish lack the postural variation, cast shadows and motion blur that characterise seabird imagery. The low score of 0.563 indicates that such scenes remain genuinely difficult even for the strongest model, yet YOLOv11 still outperformed every other model tested. The result therefore suggests, but does not confirm, that YOLOv11's spatial attention assists detection in dense penguin colonies.

2.2.5 YOLOv26

YOLOv26 (Jocher *et al.*, 2026) represents the most recent iteration of the YOLO family and targets a longstanding tension in the architecture's design: the computational overhead introduced by NMS post-processing and Distribution Focal Loss (DFL) during inference. Where YOLOv11 retains both components (Section 2.2.4), YOLOv26 removes DFL and makes NMS optional.

YOLOv26 is trained with two detection heads. The default one-to-many head predicts several boxes per object and still applies NMS, while an optional one-to-one head outputs a single box per object, producing final detections without NMS. Removing DFL also simplifies the prediction head, since each box edge no longer needs to be decoded from a probability distribution. Using the NMS-free head, the developers report up to 43% faster CPU inference for YOLOv26n than for YOLOv11n, measured with ONNX on an Intel Xeon CPU at 2.00 GHz. On this CPU, YOLOv26n took 38.9 ms per image and reached an mAP@50:95 of 40.1 on COCO, slightly below the 40.9 of the default head (Jocher *et al.*, 2026). These figures come from the developers' own benchmarks, and the only independent analysis found at the time of writing, Chakrabarty (2026), reviews this

documentation without new measurements. The 43% claim therefore awaits independent replication.

YOLOv26 also changes how it is trained. Small-Target-Aware Label Assignment (STAL) ensures that small objects still receive positive training labels, Progressive Loss shifts supervision towards the head used at inference, and the MuSGD optimiser combines SGD with the Muon optimiser (Jocher *et al.*, 2026). Although YOLOv26 itself has limited independent evaluation, both of its central design choices have support in prior work. Its NMS-free design follows YOLOv10, which achieved the highest precision, recall and F1-score among the YOLO models evaluated by Srinithi *et al.* (2024) on camera trap imagery, although YOLOv9 reached a higher mAP@50. On a Tesla T4 GPU, Minh *et al.* (2025) also found YOLOv10n faster than YOLOv11n and YOLOv12n, at 12.1 against 14.1 and 18.5 ms per image, although this advantage reversed at the large model size. Removing NMS also avoids the suppression of valid detections among overlapping individuals described in Section 2.1.4. The motivation for STAL is similarly grounded in UAV detection studies, where replacing the IoU-based box loss with one designed for small objects added 0.6 points of mAP@50 on drone images (Wang *et al.*, 2025a). YOLOv26 can therefore be understood as combining two independently supported ideas, NMS-free inference and small-target-aware training, into a single architecture suited to dense, small-object scenes such as penguin colonies.

2.3 Hardware-efficient Inference and Model Deployment

Beyond architectural design, the choice of inference engine affects real-world detection performance on resource-constrained hardware. This section reviews the main inference engines available for CPU deployment, followed by techniques that reduce computational cost by lowering numerical precision or model size.

The dominant deployment options for CPU-based inference are native framework execution (PyTorch CPU), the cross-platform ONNX Runtime, and hardware-specific toolkits such as Intel OpenVINO. Native PyTorch CPU inference requires no export step, making it the

natural baseline. It executes operations one at a time rather than optimising the model as a whole, although each operation still uses optimised CPU libraries. ONNX Runtime converts the model into a static graph and applies optimisations such as operator fusion while remaining hardware-agnostic. Alizadeh and Castor (2024) found that converting vision and language models to ONNX reduced inference time and energy use in most of the configurations they tested. Intel OpenVINO goes further by targeting Intel CPUs and integrated GPUs (iGPUs) specifically. It converts a model into its own Intermediate Representation (IR), fuses layers and selects kernels optimised for the processor's instruction set, with optional weight compression to lower precision (Intel Corporation, 2026).

Andriyanov (2020) evaluated OpenVINO against native TensorFlow execution on an Intel Core i5-4460 CPU running SSD_MobileNet_V2_COCO on 200 COCO images at 300×300 resolution. OpenVINO increased throughput by an average of $126.45\times$ in frames processed per second, with no loss in detection accuracy. This gain reflects the specific combination of model architecture, image resolution and TensorFlow as the comparison baseline, so similar gains should not be assumed against more optimised runtimes such as ONNX Runtime. Zunin (2021) further demonstrated that routing inference through the iGPU integrated into standard Intel processors can yield additional throughput over CPU-only execution, even at FP32 precision and without requiring a discrete GPU. Together, these findings suggest that platform-specific optimisation can make field deployment on standard CPU hardware more feasible.

A complementary lever for deployment efficiency is to reduce numerical precision. Full-precision FP32 is the default in most deep learning frameworks. Converting weights to 16-bit floating point (FP16) halves the memory required to store them, while INT8 quantisation, which represents weights and activations as 8-bit integers, reduces model size by a factor of four and can reduce inference latency (Jacob *et al.*, 2018). OpenVINO exposes both FP16 and INT8 execution modes natively. However, quantisation introduces a small, task-dependent loss in accuracy, which is especially consequential for ecological monitoring tasks, where each missed detection results in an uncounted individual.

Knowledge distillation provides a further approach, in which a student model is trained

to reproduce the predictions of a larger teacher model (Gou *et al.*, 2021). A simpler alternative is to use a smaller variant of the same architecture, such as nano-scale YOLO models, which trade some accuracy for reduced inference latency (Ultralytics, 2026). Since both precision reduction and knowledge distillation can affect model accuracy, establishing a full-precision baseline is necessary to assess their effects.

2.4 Datasets and Annotation Strategies

The effectiveness of object detection models is strongly dependent on the quality and representativeness of training data. While large-scale benchmarks such as COCO (Lin *et al.*, 2014) have driven progress in general object detection, ecological monitoring remains constrained by limited, domain-specific datasets characterised by complex backgrounds, variable illumination and large variation in object scale.

For African Penguins specifically, Van den Berg (2025) collected the Simple Single Penguin (SSP) dataset. This dataset contains 500 images extracted from short video sequences recorded at Boulders Beach and the Two Oceans Aquarium in Cape Town, South Africa, with each image containing a single penguin instance under varied environmental conditions. Despite its simplicity, the dataset captures variation in background complexity and lighting, providing a controlled basis for evaluating detection performance under differing scene conditions. However, because each image contains only one penguin, it does not represent the dense, overlapping scenes found in colonies.

In the broader context of penguin monitoring, the Penguin Watch project (Jones *et al.*, 2018) released 73,802 time-lapse images from 15 cameras across the Antarctic Peninsula, the South Shetland Islands and South Georgia, annotated by volunteers who placed dots on each penguin through the Zooniverse platform.[1] While this approach enables annotation at scale across multiple colonies and time periods, it introduces label noise, most of all in dense scenes where individuals are small, overlapping or partially occluded (Section 2.1.2).

[1] https://www.zooniverse.org/projects/penguintom79/penguin-watch

iNaturalist, a citizen science platform where users upload wildlife observations, contains images of multiple penguin species across diverse environmental conditions (iNaturalist, 2026). Although this diversity may improve generalisation, it also increases annotation heterogeneity and preprocessing complexity, and the images carry no bounding box annotations.

Drawing bounding boxes by hand is slow, and crowdsourced labels introduce noise that can degrade model performance if uncorrected. Semi-automated pipelines reduce this effort by combining open-vocabulary detection with promptable segmentation. In Grounded SAM (Ren *et al.*, 2024), for example, GroundingDINO locates objects from a text prompt and passes each box to SAM (Kirillov *et al.*, 2023), which returns a mask, and the authors note that human annotators can filter inaccurate predictions at each stage. SAM 3 (Carion *et al.*, 2026) performs both steps in a single model. Given a short noun phrase such as "penguin" or an example image region, it finds and segments every matching instance in an image, a task its authors call promptable concept segmentation, while still accepting the point and box prompts of earlier SAM versions. Zero-shot, SAM 3 reached a mask AP of 48.8 on the LVIS benchmark, against 38.5 for the previous best, and on the authors' own SA-Co benchmark it achieved about 74% of estimated human performance, running at 30 ms per image on an NVIDIA H200 GPU. Even so, automatically generated labels can still contain errors, so human verification remains necessary. Because annotation is performed once, before training, the computational cost of these large models does not affect deployment (Section 2.1.3).

2.5 Critical Analysis of Literature

The studies reviewed in this chapter show that automated detection can approach expert counts for penguins. Liu *et al.* (2020) came within 2% of expert counts on half of their orthomosaics, and Wu *et al.* (2023) reported 94.6% counting accuracy. Density-based (Arteta *et al.*, 2016) and open-world (Amini-Naieni *et al.*, 2024) approaches also count well, but give up explicit localisation or rely on large GPU models. Among detectors, a

YOLO-based penguin detector outperformed Faster R-CNN by 8.5 points of mAP@50:95 while running more than twice as fast (Wu *et al.*, 2023), and YOLO variants performed well on camera trap images with variable lighting and backgrounds (Srinithi *et al.*, 2024). Recent work has improved YOLO accuracy by adding modules, at a cost in computation (Wang *et al.*, 2025a,b), and improved speed by removing NMS (Minh *et al.*, 2025, Jocher *et al.*, 2026). Toolkits such as OpenVINO can also speed up inference on Intel processors (Andriyanov, 2020, Zunin, 2021).

However, almost all of these results were obtained on GPUs. The speed comparisons between YOLO versions were measured on GPUs (Wu *et al.*, 2023, Minh *et al.*, 2025), OpenVINO's reported speed-up was measured against TensorFlow (Andriyanov, 2020), and YOLOv26's reported CPU speed-up of up to 43% comes from its developers' own benchmark (Jocher *et al.*, 2026). None of the studies reviewed here measures CPU latency together with detection and counting accuracy on penguin imagery. It therefore remains unclear how much accuracy a lightweight detector loses on a CPU, and whether OpenVINO still gives a speed-up over a modern runtime such as PyTorch. This matters for field monitoring, where automated systems have run on standard laptops (Sherley *et al.*, 2010).

This study addresses that gap by comparing the nano and medium versions of YOLOv11 and YOLOv26 on a merged penguin dataset, measuring detection accuracy, counting accuracy and CPU latency in both native PyTorch and OpenVINO.

3

Methodology and Implementation

This chapter describes the methodology followed in this study and how each stage was implemented. Each section first briefly justifies the approach taken, drawing on the literature reviewed in Chapter 2, and then describes what was implemented. The chapter covers dataset construction, model selection and training, hyperparameter tuning, deployment and benchmarking, and the experiments used to address the research objectives in Section 1.4.

3.1 Methodology Overview

As noted in Section 2.5, detection models for wildlife monitoring are rarely evaluated on CPU hardware. This study therefore trains lightweight YOLO detectors on penguin imagery and benchmarks their accuracy and CPU inference speed, following the pipeline in Figure 3.1. Images from several public sources were merged into a single-class dataset and annotated with SAM 3. Four YOLO models, the nano and medium versions of YOLOv11 and YOLOv26, were then trained under identical settings. Each model was exported to OpenVINO and evaluated on detection accuracy, counting accuracy and CPU latency, with native PyTorch as the baseline.

The study used two versions of the dataset. The original split evaluated only African Penguin photographs, while a revised split, introduced to keep related photographs out

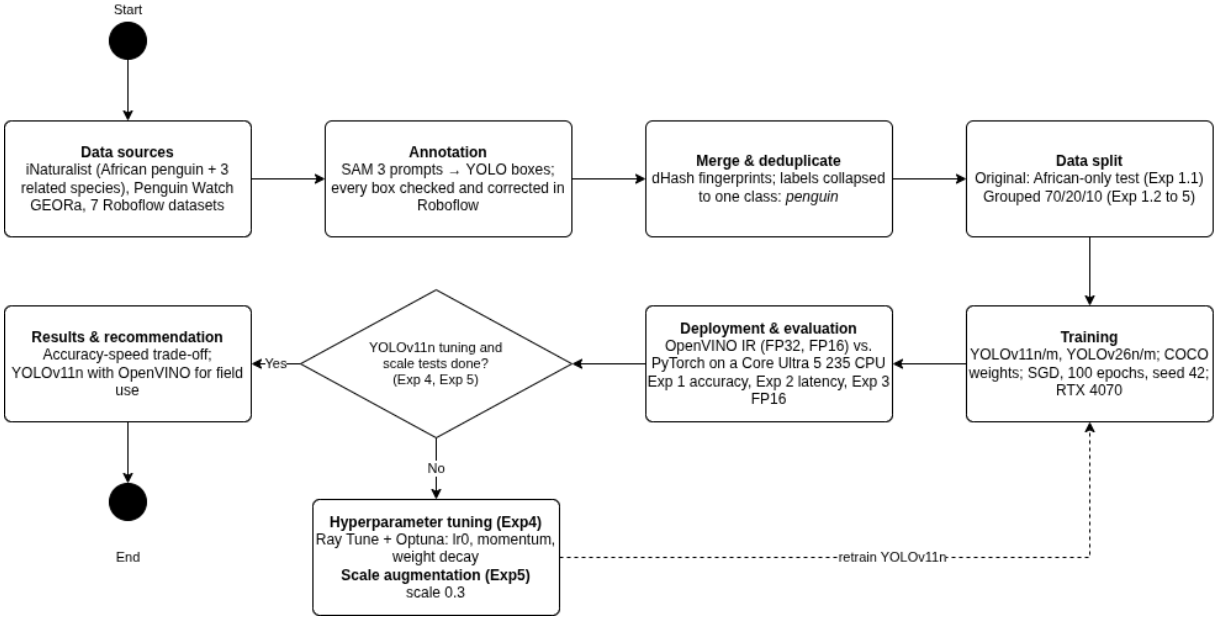


Figure 3.1: Overview of the model development methodology.

of both the training and test sets, divides all sources 70/20/10 by group (Section 3.2.4). Five experiments address the research objectives in Section 1.4. Experiment 1 measures the detection and counting accuracy on both splits (Objective 1). Experiment 2 and 3 measure CPU latency, comparing OpenVINO and PyTorch and FP32 with FP16 precision (Objective 2). Experiments 4 and 5 test whether the accuracy of YOLOv11n can be raised through hyperparameter tuning or a narrower scale augmentation, both of which change accuracy without changing inference speed (Objective 3).

3.2 Dataset Construction

No African Penguin detection dataset of sufficient size is publicly available. Schneider *et al.* (2020) found that classes with fewer than 500 training images produced unreliable recall, and that models overfit to the backgrounds of the locations they were trained on. Images from several public sources were therefore merged into one dataset, increasing both the number of penguins and the range of backgrounds seen during training.

3.2.1 Data Sources

Table 3.1 lists the sources used. The primary source was iNaturalist, from which African Penguin (*Spheniscus demersus*) photographs were collected through the Penguins of South Africa project (iNaturalist, 2026) and filtered to retain images with clearly visible penguins and sufficient quality. These photographs are taken by members of the public, so viewpoint, distance and lighting vary far more than in the aerial and fixed-camera imagery reviewed in Section 2.1.

iNaturalist photographs of the Magellanic, Humboldt and Galápagos penguins, three closely related species, add examples with similar body shape and plumage. The GEORa camera of the Penguin Watch project (Jones *et al.*, 2018) adds fixed-camera colony images containing dense, overlapping penguins at small scale, a condition largely absent from the iNaturalist photographs. These images were taken from the Oxford penguin counting dataset (Arteta *et al.*, 2016).[1] Seven further colony, aerial and close-range datasets hosted on Roboflow Universe broaden the range of backgrounds seen during training. These sets label every bird as a penguin without recording its species, shown as "Unspecified" in Table 3.1.

3.2.2 Annotation Pipeline

The iNaturalist images had no bounding boxes, the Penguin Watch images had only crowdsourced dot labels (Jones *et al.*, 2018), and the Roboflow datasets came with heterogeneous labels. The sources were therefore annotated using SAM 3 (Carion *et al.*, 2026), a promptable segmentation model that can find and segment every instance of an object described by a short text prompt. SAM 3 performs in a single model the detection and segmentation steps that the pipelines reviewed in Section 2.4 carry out separately. Images without labels were annotated from the prompt "a penguin". Where labels already existed, each crowdsourced dot, or the centre of each existing box, was converted into a small box prompt, so that SAM 3 produced one mask per labelled penguin.

[1] https://www.robots.ox.ac.uk/~vgg/data/penguins/

Table 3.1: Images contributed by each source after duplicate removal (37,487 in total). GEORa was used for training only, and all other sources were divided between training, validation and test sets (Section 3.2.4).

Source	Species	Images
iNaturalist	*S. demersus*	8 643
iNaturalist	*S. magellanicus*	12 453
iNaturalist	*S. humboldti*	4 167
iNaturalist	*S. mendiculus*	3 455
Penguin Watch, GEORa camera (Arteta *et al.*, 2016, Brown, 2026)	*Pygoscelis* spp.	3 635
penguin-seg-combined (Roboflow)	Unspecified	3 465
penguin-clfnj (new-workspace-tdyir, 2022)	Unspecified	487
penguin-water (Roboflow)	Unspecified	447
Penguin-Seaworld (Roboflow)	Unspecified	320
OCAT (Roboflow)	Unspecified	193
pingwiny-spheniscus (Roboflow)	Unspecified	190
penguin-water-hard-fp (Roboflow)	Unspecified	32

For text-prompted images, detections with a confidence below 0.25 were discarded and duplicate masks were removed using class-agnostic NMS at an IoU threshold of 0.55. In all cases, masks smaller than 324 pixels or spanning almost the full image were rejected. A tight bounding box was then taken around each remaining mask and converted to YOLO format. As SAM 3 reaches only about 74% of human performance on its author's benchmark (Section 2.4), all 37,487 images were checked in the Roboflow annotation tool. Missed penguins (false negatives) were added, and incorrect boxes (false positives) were corrected or removed.

3.2.3 Merging and Deduplication

All sources were merged into a single YOLO-format dataset using a custom Python script, and every annotation was flattened into one class, penguin. A single class was used because the conservation-relevant output is a count rather than a species label, and because even large counting models cannot reliably distinguish adults from chicks (Section 2.1.3). Each image was renamed using its source and a running number, so that files from different sources could never share a name.

Several Roboflow-hosted datasets had been exported more than once or combined into larger sets, so the same photograph could appear in several sources, sometimes resized or re-compressed. Duplicates were therefore detected with a perceptual hash, dHash, which reduces each image to a 9×8 greyscale grid and records whether each pixel is brighter than its right-hand neighbour. The resulting 64-bit fingerprint stays the same when an image is resized or saved again, and only the first image with each fingerprint was kept. This removed 6,439 images, including every image from nine Roboflow-hosted datasets, five of which were confirmed to be subsets of penguin-seg-combined. The original split used in Experiment 1.1 was built with an earlier version of the script, which compared an MD5 hash of each file's first 8 KB and therefore removed only byte-identical copies.

3.2.4 Data Split

Two splits were used, summarised in Table 3.2.

Original split. The original split, used in Experiment 1.1, kept the train, validation and test division of the African Penguin iNaturalist source and assigned all other sources to training, so that models were evaluated on African Penguins only. Its validation and test sets were therefore small, at 1,733 and 855 photographs.

Grouped split. The grouped split, used in Experiments 1.2 to 5, divides all sources except GEORa into training, validation and test sets in a 70/20/10 ratio. To keep closely related photographs on the same side of the split, images were first grouped. iNaturalist files are named by observation and photograph number, so all photographs from one observation, taken by the same person at the same place and time, formed one group. Roboflow exports keep the original file name before an added suffix (.rf), so every copy of the same original image formed one group. The groups were shuffled with a fixed seed of 42 and assigned to the three sets until each reached its share of images. GEORa images come from a single time-lapse camera, so consecutive frames are near-identical, and all of them were assigned to training. The test set contains 3,383 photographs, 835 of them of African

Penguins, so results can be reported both overall and for the target species. These 835 photographs are not a separate split but the test photographs from the African Penguin iNaturalist source, identified by the source name kept in each file name. Photographs containing penguins held between 1 and 127 penguins in training and between 1 and 102 in the test set, with a median of 2 in every set. No photograph contained more than 300 penguins, the maximum that YOLOv26's NMS-free head can output.

About 19% of the training images contain no penguins. These were kept as negative examples, a ratio close to that of Wu *et al.* (2023), who kept roughly one background image for every four images containing penguins, while Liu *et al.* (2020) also added background images to reduce false detections. Because the grouped split places such images in the test set as well, Experiment 1.2 measures false detections on the 558 test photographs without penguins.

The split separates photographs and observations but not locations, so photographs from the same beaches can still appear in both training and test sets. This is discussed as a limitation in Chapter 5.

Table 3.2: Training, validation and test sets for the two splits. Excluding the 3,635 GEORa training images, the grouped split is exactly 70/20/10.

Dataset	Set	Images	Split	Penguins	Mean per image	No penguins
Original	Train	33 357	92.8%	186 820	5.6	7 013
	Validation	1 733	4.8%	10 685	6.2	0
	Test	855	2.4%	5 692	6.7	0
Grouped	Train	27 333	72.9%	188 310	6.9	5 269
	Validation	6 771	18.1%	31 018	4.6	1 064
	Test	3 383	9.0%	15 632	4.6	558

3.3 Model Selection and Training

Four models were selected: the nano and medium variants of YOLOv11 and YOLOv26. Comparing two model sizes within two YOLO generations allows the effect of architecture to be separated from the effect of model size. As discussed in Sections 2.2.4 and 2.2.5,

YOLOv11 represents the conventional YOLO design with NMS and DFL, while YOLOv26 removes DFL, offers an NMS-free detection head and adds STAL for small objects. Its inclusion also provides an independent evaluation of a model whose reported speed-up rests on its developers' own benchmarks. Two-stage and transformer-based detectors were excluded for the reasons given in Section 2.2.2: the sequential design of two-stage detectors makes them slower, and the real-time performance of transformer-based detectors has mainly been demonstrated on GPUs.

All four models were initialised from the COCO-pretrained weights provided by Ultralytics (Ultralytics, 2026) and fine-tuned with all layers trainable, on the hardware and software listed in Table 3.3. To ensure that differences between models came only from their size and architecture, all four were trained with identical settings. Training ran for a maximum of 100 epochs at an image size of 640 pixels and a batch size of 16, with early stopping after 20 epochs without improvement in validation performance. The optimiser was set explicitly to SGD with the Ultralytics default learning rate of 0.01, momentum of 0.937 and weight decay of 0.0005. YOLOv26 was therefore trained with stochastic gradient descent (SGD) rather than its MuSGD optimiser, which keeps the comparison fair but may slightly understate its accuracy.

Training used the default Ultralytics augmentation: mosaic composition, disabled for the final 10 epochs, random scaling by a factor of 0.5 to 1.5, translation of up to 10%, horizontal flipping with probability 0.5, and random changes in hue, saturation and brightness. A fixed seed of 42 with deterministic training made the runs repeatable, and retraining YOLOv11n with the same seed reproduced its results exactly. All four models completed the full 100 epochs. The medium models reached their best validation score at epochs 96 and 93, while both nano models did so in the final epoch, so the nano models may have improved slightly with longer training. The models for Experiment 1.1 were trained in the same way, except that Ultralytics selected the optimiser automatically and YOLOv11m used a batch size of 8, an inconsistency removed in the grouped-split runs. The training scripts and full settings are available at https://github.com/hxako23/african-penguin-monitoring.

Table 3.3: Hardware and software used for training and benchmarking.

Role	Component	Specification
Training	GPU	NVIDIA RTX 4070 (12 GB GDDR6X)
Training	Software	Python 3.13, PyTorch 2.11.0 (CUDA 13.0), Ultralytics 8.4.41
Benchmarking	CPU	Intel Core Ultra 5 235, 14 threads
Benchmarking	Inference	OpenVINO 2026.2.0 (CPU device) and native PyTorch
Benchmarking	Precision	FP32, and FP16 for OpenVINO

3.4 Hyperparameter Tuning and Scale Augmentation

Experiments 4 and 5 test whether the accuracy of YOLOv11n, the primary lightweight candidate, can be raised without changing the model. Neither tuning nor augmentation alters the architecture, so CPU latency stays the same, and any gain in accuracy would narrow the gap to the medium models at no cost in speed. This addresses the accuracy-speed trade-off of Objective 3.

Hyperparameter tuning. The search covered the initial learning rate, momentum and weight decay across ten trials, using Ray Tune (Liaw *et al.*, 2018) with the Optuna search algorithm (Akiba *et al.*, 2019), which uses the results of earlier trials to select promising values for later ones. The learning rate was sampled between 5×10^{-4} and 5×10^{-3} and weight decay between 10^{-5} and 10^{-3}, both on a logarithmic scale, and momentum uniformly between 0.85 and 0.98. Each trial continued training the baseline YOLOv11n for 20 epochs, rather than starting from the COCO weights, using the training recipe of the tuning script. This recipe differs from the baseline in more than the three searched values. It uses the AdamW optimiser instead of SGD and a seed of 0, keeps mosaic augmentation for all but the last two epochs of each trial, scales images by a factor of 0.8 to 1.2 instead of 0.5 to 1.5, and rotates them by up to 10 degrees. It also replaces the Ultralytics colour changes with an additional augmentation pipeline that applies, with low probability, effects such as fog, rain, sensor noise, blur and JPEG compression. The batch size of 16 matched the baseline. The best setting was then used to continue training the baseline for a further 20 epochs. Trials were scored on validation mAP@50:95, which

rewards tighter boxes than mAP@50, and the test set was used only to evaluate this final model. Because every trial continued from the already trained baseline under this recipe, the experiment tests whether this recipe can improve a trained model, rather than the effect of the three searched values on their own. Comparing trials with each other still indicates how these values affect accuracy within the recipe.

Scale augmentation. During training, Ultralytics randomly resizes each image by a factor controlled by its scale setting, which defaults to 0.5 and gives factors between 0.5 and 1.5. This teaches the model to recognise penguins at different sizes, an approach also used by Arteta *et al.* (2016), who trained on each image at six scales. Experiment 5 retrained YOLOv11n from the COCO weights with a scale setting of 0.3, giving factors between 0.7 and 1.3, while keeping every other setting identical to the baseline in Section 3.3, including the seed. Any difference from the baseline can therefore be attributed to the narrower scale range. Both models were evaluated in the same way as in Experiment 1.2.

3.5 Deployment and Benchmarking

Of the CPU inference options reviewed in Section 2.3, OpenVINO was selected because it is designed specifically for Intel processors, such as the one used for benchmarking, with native PyTorch as the baseline. Each trained model was exported to the OpenVINO Intermediate Representation with a fixed input size of 640×640 pixels, once at full FP32 precision and once with its weights compressed to FP16. YOLOv26 models used their NMS-free one-to-one head in both runtimes, which outputs up to 300 final detections per image, while YOLOv11 models apply NMS after inference. All models were run on the CPU device, and the processor's integrated GPU was not evaluated because of time constraints.

To make the comparison between runtimes fair, every test photograph was first resized and padded to 640×640 pixels, so that OpenVINO and PyTorch processed identical inputs. Without this step, PyTorch pads each photograph only to the next multiple of 32 pixels,

while the OpenVINO models always process the full fixed input. Both runtimes were timed back to back in the same session, with no other sessions running. Ten warm-up inferences were performed before timing to exclude model loading and compilation. Each of the 3,383 test photographs were then processed individually at batch size 1, matching single-frame processing in the field, and the time for each prediction was recorded, including image loading, pre-processing, inference and post-processing. The mean and standard deviation of latency per image and the frames per second were recorded for both runtimes. PyTorch used 14 CPU threads, and OpenVINO ran in its latency mode for single-image inference. The FP16 models were timed in a second session alongside their FP32 counterparts. The benchmarking scripts are available in the same repository.

3.6 Experimental Setup

Five experiments were defined to address the research objectives in Section 1.4. Except in Experiment 4, detection accuracy was measured using the OpenVINO models on the CPU, which reflects the deployed models. Counting used a confidence threshold of 0.25, the Ultralytics default.

Experiment 1: Detection and counting accuracy. This experiment tests the accuracy part of the hypothesis: whether the nano models stay within 5 points of the medium models on mAP@50 and within 5% of the true count. Experiment 1.1 evaluates the models trained on the original split, on its 855 African Penguin test photographs. Experiment 1.2 evaluates the models trained on the grouped split, on all 3,383 test photographs and separately on the 835 African Penguin photographs among them, and also counts false detections on the 558 test photographs without penguins.

Experiment 2: CPU latency. This experiment tests the latency part of the hypothesis, that OpenVINO reduces CPU latency compared with native PyTorch, and reports how many images per second each model processes. It uses the timing procedure in Section 3.5 on the grouped split.

Experiment 3: Numerical precision. This experiment compares the FP32 and FP16 OpenVINO models on model size, detection accuracy and latency, to test whether lower precision reduces latency on this CPU without reducing accuracy.

Experiment 4: Hyperparameter tuning. Using the procedure in Section 3.4, this experiment reports the spread of validation mAP@50:95 across the ten trials and the test accuracy of the final tuned YOLOv11n, on the original split (Experiment 4.1) and the grouped split (Experiment 4.2).

Experiment 5: Scale augmentation. This experiment compares YOLOv11n trained with scale settings of 0.5 and 0.3, using the same detection and counting evaluation as Experiment 1.2. Latency was not measured, as the architecture is unchanged.

Metrics. Detection accuracy is reported as mAP@50, which counts a box as correct when it overlaps a labelled box by at least half, and mAP@50:95, which averages over stricter overlaps and therefore rewards tighter boxes. Precision and recall are reported at the confidence that maximises F1, as computed by Ultralytics. Because mAP is calculated across all confidence thresholds, it does not depend on the threshold used for counting. Counting accuracy is reported as the mean absolute error (MAE) between the predicted and true count for each photograph containing penguins, the share of these photographs counted within 5% of the true count, and the error in the total count over the test set. As most test photographs contain only one or two penguins, a single miscounted penguin already exceeds 5%, so this share mostly measures how often the count is exactly right. Latency is reported as the mean and standard deviation per image in milliseconds, and as frames per second.

3.7 Summary

This chapter described how images from several public sources were merged into a single-class penguin dataset, annotated with SAM 3, and divided into an original and a grouped split. It then described how four YOLO models were trained under identical settings, how YOLOv11n was tuned and retrained with a narrower scale augmentation, and how all models were exported to OpenVINO at FP32 and FP16 and benchmarked against native PyTorch on a CPU. Chapter 4 presents the results of the five experiments.

4

Results and Discussion

This chapter presents the results of the five experiments described in Chapter 3, and discusses them in relation to the research objectives and the literature reviewed in Chapter 2.

4.1 Experiment 1: Detection and Counting Accuracy

4.1.1 Experiment 1.1: Original Split

Table 4.1 summarises the results on the original split, whose 855 test photographs show African Penguins only. YOLOv26m was the most accurate model, with an mAP@50 of 0.928 and an mAP@50:95 of 0.744, 2.0 and 4.0 points above YOLOv11m. The nano models scored 2.0 points (YOLOv11) and 4.4 points (YOLOv26) below their medium counterparts on mAP@50, inside the 5-point margin set in Section 1.3. Every model counted with an MAE below one penguin per photograph, and 67.6 to 70.8% of photographs were counted within 5% of the true count.

4.1.2 Experiment 1.2: Grouped Split

Detection. Table 4.2 reports detection accuracy on the grouped split, for all 3,383 test photographs and for the 835 African Penguin photographs among them. The medium

Table 4.1: Experiment 1.1: detection and counting accuracy on the original split (855 African Penguin test photographs). Detection values were measured with the OpenVINO FP32 models on the CPU. These models were trained with the optimiser chosen automatically by Ultralytics and, for YOLOv11m, a batch size of 8 (Section 3.3), and were tested on a different test set, so their results are not directly comparable with Table 4.2.

Model	mAP@50	mAP@50:95	MAE	Within 5%
YOLOv11n	0.8882	0.6648	0.88	**70.8%**
YOLOv26n	0.8838	0.6674	0.94	68.2%
YOLOv11m	0.9077	0.7043	0.90	67.6%
YOLOv26m	**0.9277**	**0.7440**	**0.72**	67.6%

models were again the most accurate, but the lead of YOLOv26m over YOLOv11m shrank to 0.1 points on mAP@50 and 1.0 point on mAP@50:95, compared with 2.0 and 4.0 points on the original split. At nano size, the two generations were level, within one point on both metrics. As each model was trained once, differences of around one point may vary between runs, so neither generation was clearly more accurate at the same size. Model size had the larger effect: moving from medium to nano reduced mAP@50 by 3.8 points for YOLOv11 and 4.6 points for YOLOv26, inside the 5-point margin, and mAP@50:95 by 7.1 and 8.0 points, so the medium models also placed their boxes more tightly. Every model scored 2.8 to 3.6 points higher on the African Penguin photographs than on the full test set, which also contains other species and colony and aerial images in which penguins are often smaller.

Table 4.2: Experiment 1.2: detection accuracy on the grouped split, measured with the OpenVINO FP32 models on the CPU. Precision and recall are given at the confidence that maximises F1.

Test set	Model	mAP@50	mAP@50:95	Precision	Recall
All (3,383)	YOLOv11n	0.8616	0.6568	0.8809	0.7710
	YOLOv26n	0.8543	0.6572	0.8702	0.7618
	YOLOv11m	0.8996	0.7276	0.8913	**0.8361**
	YOLOv26m	**0.9007**	**0.7376**	**0.9012**	0.8228
African Penguin (835)	YOLOv11n	0.8974	0.6713	0.9078	0.8148
	YOLOv26n	0.8884	0.6697	0.8940	0.8003
	YOLOv11m	0.9271	0.7434	0.9238	**0.8726**
	YOLOv26m	**0.9308**	**0.7564**	**0.9324**	0.8592

Counting: Table 4.4 reports counting accuracy on the photographs containing penguins, together with false detections on the 558 photographs without penguins. Three of the four models counted with an MAE below one penguin per photograph, and 66.6 to 70.4% of photographs were counted within 5%. With a median of two penguins per photograph, a single miscounted penguin already exceeds 5%, so this share mostly measures how often the count was exactly right. Summed over the whole test set, the medium models came within about 1% of the true total of 15,632 penguins, and within 0.4% on African Penguins, meeting the 5% target at census level. The nano models undercounted, by 5.4% (YOLOv11n) and 11.1% (YOLOv26n), consistent with their lower recall in Table 4.2. On African Penguins alone, YOLOv11n came within 1.9% of the true total, while YOLOv26n remained 6.7% below it. These totals are comparable in kind to the 2% agreement with expert counts reported by Liu *et al.* (2020), although that study counted nests in drone orthomosaics. On the 558 photographs without penguins, the models produced 232 to 300 false boxes. These false boxes are not part of the totals, which cover only the photographs containing penguins, but they show that the medium models detect more readily than the nano models. With a recall of 0.82 to 0.84 (Table 4.2), the medium models still missed some penguins, yet their total counts were close to the true totals (Table 4.4). This suggests that the missed penguins were partly cancelled out by extra boxes on photographs that do contain penguins. The confidence threshold of 0.25 was also not tuned for counting, although Wu *et al.* (2023) showed that this choice affects counting accuracy.

Table 4.3 splits the counting results by the number of penguins per photograph. On the 1,763 photographs with one or two penguins, every model counted most photographs exactly, with an MAE below 0.2, so the models differed little. Errors grew with crowding: on the 336 photographs with more than 10 penguins, only 18 to 29% were counted within 5%, and the nano models undercounted by 7.5% (YOLOv11n) and 15.8% (YOLOv26n), against about 3% for the medium models. The undercounting of the nano models in Table 4.4 therefore comes mainly from the crowded photographs. YOLOv26 showed no advantage in this group, as YOLOv26n had the largest error and YOLOv26m was level with YOLOv11m, so these results do not support the suggestion that its small-target training helps in dense scenes.

Table 4.3: Counting accuracy by the number of penguins per test photograph, on the grouped split at a confidence threshold of 0.25. Total error is the difference between the predicted and true totals within each group, with negative values showing undercounts. The same results are shown as a figure in Appendix A.2.

Penguins per photo	Model	MAE	Within 5%	Total error
1 to 2 (1,763 photos)	YOLOv11n	0.17	86.0%	+1.0%
	YOLOv11m	0.16	87.2%	+3.9%
	YOLOv26n	0.18	85.3%	+1.3%
	YOLOv26m	0.18	86.2%	+2.5%
3 to 10 (726 photos)	YOLOv11n	1.08	45.7%	−3.8%
	YOLOv11m	0.94	48.8%	+1.1%
	YOLOv26n	1.10	43.7%	−5.7%
	YOLOv26m	0.98	47.8%	+1.8%
Over 10 (336 photos)	YOLOv11n	4.67	22.9%	−7.5%
	YOLOv11m	3.74	29.2%	−3.1%
	YOLOv26n	5.80	18.2%	−15.8%
	YOLOv26m	3.85	27.7%	−2.9%

Table 4.4: Experiment 1.2: counting accuracy on the grouped split at a confidence threshold of 0.25. MAE, within 5% and total error use the photographs containing penguins: 2,825 in the full test set and all 835 African Penguin photographs. Total error is the difference between the predicted and true totals, with negative values showing undercounts. False boxes are detections on the 558 test photographs without penguins, none of which are African Penguin photographs.

Test set	Model	MAE	Within 5%	Total error	False boxes
All	YOLOv11n	0.94	68.1%	−5.4%	265
	YOLOv26n	1.09	66.6%	−11.1%	**232**
	YOLOv11m	**0.79**	**70.4%**	−1.1%	300
	YOLOv26m	0.82	69.3%	−1.0%	296
African	YOLOv11n	0.87	68.3%	−1.9%	–
	YOLOv26n	1.03	66.5%	−6.7%	–
	YOLOv11m	**0.69**	**70.5%**	**+0.2%**	–
	YOLOv26m	0.72	69.5%	+0.4%	–

4.2 Experiment 2: CPU Inference Latency

Table 4.5 reports the mean latency per image of each model on the Intel Core Ultra 5 235, measured end to end on the 3,383 test photographs, with both runtimes timed in the same session on identical 640×640 inputs.

Table 4.5: Experiment 2: mean CPU latency per image (± standard deviation) at FP32 on the grouped-split test set. Speed-up is PyTorch latency divided by OpenVINO latency.

	Latency (ms)			FPS	
Model	**PyTorch**	**OpenVINO**	**Speed-up**	**PyTorch**	**OpenVINO**
YOLOv11n	23.68 ± 0.92	15.90 ± 0.90	1.49×	42.2	62.9
YOLOv26n	23.02 ± 0.42	**14.77 ± 1.06**	**1.56×**	43.4	**67.7**
YOLOv11m	148.53 ± 2.43	111.01 ± 2.67	1.34×	6.7	9.0
YOLOv26m	149.38 ± 1.57	111.46 ± 2.70	1.34×	6.7	9.0

OpenVINO was faster than native PyTorch for every model, reducing latency by 25 to 36%, a speed-up of 1.34 to 1.56 times. This supports the direction of the hypothesis in Section 1.4, but not its size, as OpenVINO did not halve latency for any model. The speed-up was larger for the nano models than for the medium models. It is also far below the 126-fold gain reported by Andriyanov (2020), which was measured against TensorFlow rather than a modern runtime such as PyTorch, which already uses optimised CPU libraries (Section 2.3). Because latency was measured end to end, image loading and pre-processing, which cost the same for both runtimes, further reduce the relative difference between them.

These results differ from an earlier benchmark on the original split, in which PyTorch appeared faster for the medium models. In that benchmark, the two runtimes were timed in separate sessions, and PyTorch received photographs padded only to the next multiple of 32 pixels, while OpenVINO always processed the full 640×640 input, about a third more pixels for a typical $4 : 3$ photograph. Timing both runtimes together on identical inputs removed this difference and also reduced the variation between images, with standard deviations of 0.4 to 2.7 ms. A second session, run as part of Experiment 3, reproduced the OpenVINO FP32 latencies to within 4%.

With OpenVINO, the nano models processed 63 to 68 photographs per second, above the real-time threshold of 25 to 30 frames per second of standard video cameras, while the medium models processed about 9. YOLOv26n was 7% faster than YOLOv11n with OpenVINO (14.77 against 15.90 ms) and 3% faster with PyTorch, while the two medium models were within 1% of each other. This is far below the speed-up of up to 43% that

the developers of YOLOv26 report for its NMS-free head (Jocher *et al.*, 2026), measured with ONNX on a different CPU. With a median of two and a mean of 4.6 penguins per test photograph, NMS has few boxes to filter, which may explain why removing it saved little time here.

4.3 Experiment 3: Numerical Precision

Table 4.6 compares the OpenVINO models at FP32 and FP16 precision, timed in the same session on the 3,383 grouped-split test photographs.

Table 4.6: Experiment 3: OpenVINO models at FP32 and FP16 precision on the grouped-split test set. Latency is the mean per image.

	Size (MB)		**mAP@50:95**		**Latency (ms)**	
Model	**FP32**	**FP16**	**FP32**	**FP16**	**FP32**	**FP16**
YOLOv11n	10.7	5.6	0.6568	0.6567	15.32	15.54
YOLOv26n	10.0	5.3	0.6572	0.6572	14.81	15.02
YOLOv11m	80.6	40.6	0.7276	0.7275	110.59	109.41
YOLOv26m	81.9	41.3	0.7376	0.7375	110.81	110.37

FP16 roughly halved the size of every model, from 80.6 to 40.6 MB for YOLOv11m, for example, while mAP@50:95 changed by at most 0.0001, and mAP@50 and the African Penguin results showed the same pattern. FP16 gave no speed-up, however: every FP16 latency was within 1.5% of its FP32 counterpart, which is smaller than the variation between sessions. This is likely because this CPU cannot calculate in FP16 directly, so it converts the weights back to FP32 before using them (Intel Corporation, 2026). FP16 therefore halved the storage the models need, without making them any faster or less accurate. INT8 quantisation, which stores weights as 8-bit integers that CPUs can calculate with directly, remains the more promising way to reduce latency (Section 2.3).

4.4 Experiment 4: Hyperparameter Tuning

Table 4.7 compares the tuning runs with the baseline YOLOv11n on the validation set, on which the trials were scored, together with test results where available. As described in Section 3.4, every trial continued training the already trained baseline under a different recipe, so the experiment tests whether that recipe can improve a trained model.

Table 4.7: Experiment 4: hyperparameter tuning of YOLOv11n. The tuned model on the grouped split was evaluated by the tuning script with PyTorch.

Split	Run	Val mAP@50:95	Test mAP@50:95
Original (4.1)	Baseline	**0.6815**	0.6648
	10 tuning trials	0.6706 to 0.6790	–
	Best setting, retrained	0.6796	–
Grouped (4.2)	Baseline	**0.6651**	**0.6568**
	10 tuning trials	0.6214 to 0.6397	–
	Best setting, retrained	0.6406	0.6380

On the original split (Experiment 4.1), all ten trials scored between 0.671 and 0.679, below the baseline's 0.682, and retraining with the best setting (a learning rate of 0.0005, momentum of 0.950 and weight decay of 3.1×10^{-5}) reached 0.680. The retrained model scored higher than every trial, so the spread between trials, under one point, was about as large as the variation between runs, and these trials show no clear effect of the searched values.

On the grouped split (Experiment 4.2), the trials spread more widely, from 0.621 to 0.640, all 2.5 to 4.4 points below the baseline's 0.665. Within the trials, the scores generally fell as the learning rate increased from 0.640 at the lowest rate tested (0.00055) to 0.621 at the highest (0.0045), while momentum and weight decay showed no clear effect. Continuing the baseline with the best setting gave a validation mAP@50:95 of 0.641 and a test mAP@50:95 of 0.638, 2.5 and 1.9 points below the baseline. Every trial and the retrained model started from the baseline and scored below it, so the tuning recipe lowered the accuracy of an already trained model, whatever values were searched. The same happened on the original split, where the drop was smaller, at under 1.1 points. Continuing training with a different optimiser and augmentation can therefore reduce the accuracy of a trained model. The

experiment does not show whether better values of the three hyperparameters exist, and tuning in this form offered no way to narrow the gap to the medium models.

4.5 Experiment 5: Scale Augmentation

Table 4.8 compares YOLOv11n trained with the default scale setting of 0.5 and with the narrower setting of 0.3, with every other setting unchanged.

Table 4.8: Experiment 5: YOLOv11n trained with two scale settings, evaluated on the full grouped-split test set. Detection was measured with OpenVINO FP32 on the CPU, and counting at a confidence threshold of 0.25.

Scale	mAP@50	mAP@50:95	MAE	Within 5%	Total error	False boxes
0.5 (default)	**0.8616**	**0.6568**	**0.94**	**68.1%**	−5.4%	265
0.3	0.8596	0.6521	0.96	67.9%	−3.7%	266

Narrowing the scale range did not improve detection. mAP@50 fell by 0.2 points and mAP@50:95 by 0.5 points on the full test set, and by 0.3 and 0.6 points on African Penguins, so the larger loss on the stricter metric suggests slightly less precise boxes. Counting per photograph was also unchanged, with an MAE of 0.96 against 0.94, almost the same share of photographs within 5% and the same number of false detections. The one notable difference was in the total count, where the undercount fell from 5.4 to 3.7%, inside the 5% target. With an unchanged MAE and a single run of each model, however, this difference cannot be attributed to the scale setting with confidence. The test set ranges from close-up photographs of a single penguin to colony images with over 100, so the wider default range may better match the variation in penguin size that the model must handle, which is also why Arteta *et al.* (2016) trained on several scales. The default setting was therefore kept.

4.6 Accuracy-Efficiency Trade-off

Figure 4.1 plots detection accuracy against CPU latency for every model and runtime. Accuracy was measured with OpenVINO, and the PyTorch points use the same weights. Moving from a medium to a nano model made each photograph about seven times faster to process with OpenVINO, 15 to 16 instead of 111 ms, at a cost of 7.1 to 8.0 points of mAP@50:95 and 3.8 to 4.6 points of mAP@50. For a survey of 1,000 photographs, this is the difference between about 16 seconds and about 2 minutes. Switching from PyTorch to OpenVINO reduced the latency of every model by 25 to 36% at no cost in accuracy, so OpenVINO is the better runtime regardless of model size.

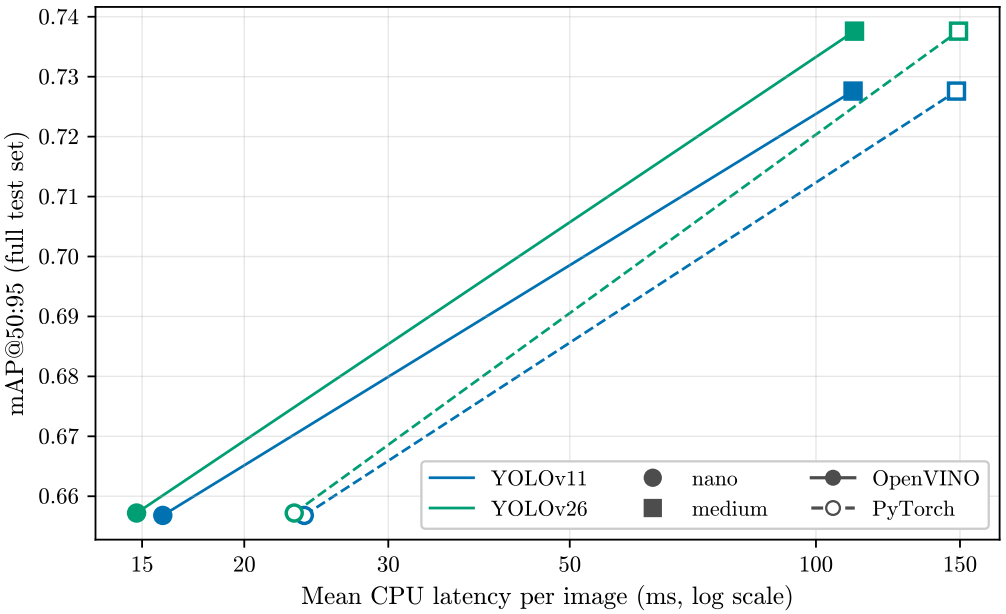


Figure 4.1: Detection accuracy against mean CPU latency per image on the grouped-split test set. Filled points use OpenVINO FP32 and hollow points native PyTorch, with accuracy measured using OpenVINO. The latency axis is logarithmic.

On detection accuracy and latency alone, YOLOv26n would be the best nano model, as it was slightly faster than YOLOv11n with the same mAP@50:95. The counting results reverse this choice. YOLOv11n undercounted the full test set by 5.4% and African Penguins by 1.9%, against 11.1% and 6.7% for YOLOv26n, consistent with its higher recall. Since the conservation-relevant output is the count, YOLOv11n with OpenVINO is recommended for field use on a standard CPU, where it processes about 63 photographs per second.

Where photographs are processed after a survey rather than as they are captured, speed matters less and a medium model is preferable. YOLOv11m came within 1.1% of the true total over the full test set and within 0.2% on African Penguins, while still processing about nine photographs per second. Neither YOLO generation offered a consistent advantage at the same size, and the 7% speed gain of YOLOv26n over YOLOv11n fell well short of the 43% reported by its developers (Section 4.2).

4.7 Summary

On both splits, the medium models were the most accurate, and the nano models stayed within 5 points of them on mAP@50, losing 3.8 to 4.6 points on the grouped split. The clear lead of YOLOv26m over YOLOv11m on the original split almost disappeared on the grouped split, so neither YOLO generation was clearly more accurate at the same size. Every model counted about two thirds of test photographs within 5%, mostly because a single miscounted penguin already exceeds 5% in a photograph with one or two penguins. Summed over the test set, the medium models came within about 1% of the true total, while the nano models undercounted, by 5.4% for YOLOv11n and 11.1% for YOLOv26n. OpenVINO was faster than native PyTorch for every model, by 1.34 to 1.56 times, and the nano models processed 63 to 68 photographs per second on the CPU. YOLOv26n was only 7% faster than YOLOv11n, far below the 43% reported by its developers. FP16 halved model size without changing accuracy or latency. Continuing YOLOv11n's training with the tuning recipe lowered its accuracy on both splits, and narrowing the scale augmentation from 0.5 to 0.3 lowered mAP@50:95 by 0.5 points, although it reduced the total undercount from 5.4 to 3.7%.

In answer to the hypothesis, the nano models stayed within 5 points of the medium models on mAP@50, but the 5% counting target was met over the whole test set only by the medium models, and for African Penguins also by YOLOv11n. OpenVINO reduced latency by about a third rather than by half. For field use on a standard CPU, YOLOv11n offers the best balance, running at about 63 photographs per second while counting African

Penguins to within 1.9% of the true total, as discussed in Section 4.6.

5

Conclusion and Future Work

This chapter answers the research objectives and states the contributions, limitations and future directions of the study.

5.1 Conclusion

As described in Chapter 1, published penguin detectors approach expert counts, but only on GPU hardware, so it was unclear how well lightweight detectors perform on the standard computers available to field teams. This study merged penguin images from several public sources into a single-class dataset of 37,487 images, labelled with SAM 3 and checked by hand, and trained the nano and medium versions of YOLOv11 and YOLOv26 under identical settings. Each model was evaluated on detection accuracy, counting accuracy and CPU latency, with OpenVINO and native PyTorch on an Intel Core Ultra 5 235. The results answer the three research objectives as follows.

1. **Accuracy (Objective 1): mAP target met, counting target partly met.** The nano models stayed within 5 points of the medium models on mAP@50, losing 3.8 to 4.6 points. The 5% target on the total count was met by the medium models, which came within about 1% of the true total, and by YOLOv11n on African Penguins (1.9%), but not by the nano models over the whole test set, which undercounted by

5.4% (YOLOv11n) and 11.1% (YOLOv26n). Per photograph, no model met the 5% target: about two thirds of photographs were counted within 5%, mainly because a single miscounted penguin exceeds 5% in the many photographs with only one or two penguins.

2. **CPU latency (Objective 2): OpenVINO faster, but the hypothesised halving not met.** OpenVINO was faster than native PyTorch for every model, by 1.34 to 1.56 times. With OpenVINO, the nano models processed 63 to 68 photographs per second, meeting the real-time definition in Chapter 1, while the medium models processed about 9. FP16 halved model size without changing accuracy or latency.

3. **Trade-off (Objective 3): measured, but not narrowed.** Moving from a medium to a nano model made inference about seven times faster at a cost of 7.1 to 8.0 points of mAP@50:95. Neither continued training with the tuning recipe, which lowered accuracy, nor a narrower scale augmentation narrowed this gap.

Two findings qualify claims from the literature. The lead of YOLOv26m over YOLOv11m on the original split almost disappeared on the stricter grouped split, and YOLOv26n was only 7% faster than YOLOv11n, far below the 43% reported by its developers.

Lightweight detection on a standard CPU is therefore feasible for African Penguin monitoring. YOLOv11n with OpenVINO is recommended for field use, running at about 63 photographs per second and counting African Penguins to within 1.9% of the true total. When photographs are processed after a survey, a medium model such as YOLOv11m gives more accurate totals at about 9 photographs per second.

5.2 Contributions

This study contributes a CPU benchmark of current YOLO models on penguin imagery that reports counting accuracy alongside detection accuracy, with both runtimes timed in the same session on identical inputs. It also provides an independent test of the

CPU speed-up reported for YOLOv26, and a dataset construction method that removes near-duplicate images and keeps related photographs on the same side of the split.

5.3 Limitations

Each model was trained once, so differences of about one point may fall within run-to-run variation. The grouped split separates observations but not locations, and the test set contains few dense scenes, with a median of two penguins per photograph, so results for crowded colonies are less certain. The nano models also undercounted most in the most crowded photographs (Table 4.3), so dense colonies are likely to be counted less accurately than the overall results suggest. Totals were summed over individual photographs rather than a surveyed colony. YOLOv26 was trained with SGD rather than its MuSGD optimiser, the tuning trials used a different recipe from the baseline, and the confidence threshold was not tuned for counting. Finally, latency was measured on a single CPU, at FP32 and FP16 only. All labels came from SAM 3, which reaches about 74% of human performance, and although all 37,487 images were checked by hand, they were not checked by independent experts, so some labelling errors may remain. The code for all experiments is publicly available at https://github.com/hxako23/african-penguin-monitoring.

5.4 Future Work

Repeating each run with several random seeds would show which differences between models are real. This was not carried out due to time and compute constraints. Furthermore, INT8 quantisation and the processor's integrated GPU (Zunin, 2021) may reduce latency further. Knowledge distillation, in which YOLOv11n learns to imitate a medium model, could narrow the accuracy gap without changing its speed, and tuning the confidence threshold for counting (Wu *et al.*, 2023) may reduce its undercounting. Finally, testing on lower-power hardware such as a Raspberry Pi (Minh *et al.*, 2025) would show whether the nano models remain fast enough in the field.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631. 2019.

Ali, M. L. and Zhang, Z. The YOLO framework: A comprehensive review of evolution, applications, and benchmarks in object detection. *Computers*, 13(12):336, 2024.

Alizadeh, N. and Castor, F. Green AI: A Preliminary Empirical Study on Energy Consumption in DL Models Across Different Runtime Infrastructures. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI*, pages 134–139. 2024. doi:10.1145/3644815.3644967.

Amini-Naieni, N., Han, T., and Zisserman, A. Countgd: Multi-modal open-world counting. *Advances in Neural Information Processing Systems*, 37:48810–48837, 2024.

Andriyanov, N. Analysis of the Acceleration of Neural Networks Inference on Intel Processors Based on OpenVINO Toolkit. In *2020 Systems of Signal Synchronization, Generating and Processing in Telecommunications (SYNCHROINFO)*, pages 1–5. IEEE, 2020.

Arteta, C., Lempitsky, V., and Zisserman, A. Counting in the wild. In *European conference on computer vision*, pages 483–498. Springer, 2016.

Bhatt, D., Patel, C., Talsania, H., Patel, J., Vaghela, R., Pandya, S., Modi, K., and Ghayvat, H. CNN variants for computer vision: History, architecture, application, challenges and future scope. *Electronics*, 10(20):2470, 2021.

BirdLife International. *Spheniscus demersus*. The IUCN Red List of Threatened Species 2024: e.T22697810A256021744. 2024. doi:10.2305/IUCN.UK.2024-2.RLTS.T22697810A256021744.en. Accessed on 6 March 2026.

Brown, D. GEORa Dataset. https://universe.roboflow.com/danesegmentation/geora, 2026. Roboflow Universe, version 1, curated from the Oxford penguin counting dataset, MIT licence, accessed 12 October 2026.

Carion, N., Gustafson, L., Hu, Y.-T., Debnath, S., Hu, R., Suris, D., Ryali, C., Alwala, K. V., Khedr, H. *et al.* SAM 3: Segment Anything with Concepts. In *International Conference on Learning Representations (ICLR)*. 2026.

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., and Zagoruyko, S. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020.

Chakrabarty, S. YOLO26: An Analysis of NMS-Free End to End Framework for Real-Time Object Detection. *arXiv preprint arXiv:2601.12882*, 2026.

Edney, A. J. and Wood, M. J. Applications of digital imaging and analysis in seabird monitoring and research. *Ibis*, 163(2):317–337, 2021.

Gao, G., Gao, J., Liu, Q., Wang, Q., and Wang, Y. A survey of deep learning methods for density estimation and crowd counting. *Vicinagearth*, 2(1):2, 2025.

Geldenhuys, F. Using species distribution models for spatial conservation planning of African penguins. Ph.D. thesis, Stellenbosch: Stellenbosch University, 2018.

Gou, J., Yu, B., Maybank, S. J., and Tao, D. Knowledge distillation: A survey. *International journal of computer vision*, 129(6):1789–1819, 2021.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778. 2016.

Hosang, J., Benenson, R., and Schiele, B. Learning non-maximum suppression. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4507–4515. 2017.

iNaturalist. Penguins of South Africa Project. 2026.
URL https://www.inaturalist.org/projects/penguins-of-south-africa

Intel Corporation. OpenVINO Toolkit Documentation. https://docs.openvino.ai/, 2026. Accessed: 1 October 2026.

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2018.

Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F. C., and Kalfaoglu, M. E. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. 2026.
URL https://arxiv.org/abs/2606.03748

Jones, F. M., Allen, C., Arteta, C., Arthur, J., Black, C., Emmerson, L. M., Freeman, R., Hines, G., Lintott, C. J., Macháčková, Z. *et al.* Time-lapse imagery and volunteer classifications from the Zooniverse Penguin Watch project. *Scientific data*, 5(1):180124, 2018.

Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y. *et al.* Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026. 2023.

Liaw, R., Liang, E., Nishihara, R., Moritz, P., Gonzalez, J. E., and Stoica, I. Tune: A Research Platform for Distributed Model Selection and Training. *arXiv preprint arXiv:1807.05118*, 2018.

Lin, T.-Y., Dollar, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. Feature Pyramid Networks for Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. July 2017.

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft COCO: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., and Berg, A. C. SSD: Single Shot MultiBox Detector. In **Leibe, B., Matas, J., Sebe, N., and Welling, M.**, editors, *Computer Vision – ECCV 2016*, pages 21–37. Springer International Publishing, Cham, 2016. ISBN 978-3-319-46448-0.

Liu, Y., Shah, V., Borowicz, A., Wethington, M., Strycker, N., Forrest, S., Lynch, H., and Singh, H. Towards efficient machine learning methods for penguin counting in unmanned aerial system imagery. In *2020 IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*, pages 1–7. IEEE, 2020.

Minh, P. Q., Tan, L. N., Tu, N. H., and Anh, N. T. Fruit Ripeness Detection: A Comparative Analysis of State-of-the-Art YOLO Models. In *2025 RIVF International Conference on Computing and Communication Technologies (RIVF)*, pages 699–704. IEEE, 2025.

new-workspace-tdyir. penguin Dataset. https://universe.roboflow.com/new-workspace-tdyir/penguin-clfnj, 2022. Roboflow Universe, version 1, MIT licence, accessed 12 October 2026.

Phan, D. T., Gao, J., Lu, Y., Yap, K.-H., Garg, K., and Han, B. S. DEO-Net: Joint Density Estimation and Object Detection for Crowd Counting. *IEEE Transactions on Instrumentation and Measurement*, 73:1–11, 2024. doi:10.1109/TIM.2024.3441018.

Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788. 2016.

Ren, S., He, K., Girshick, R., and Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015.

Ren, T., Liu, S., Zeng, A., Lin, J., Li, K., Cao, H., Chen, J., Huang, X., Chen, Y., Yan, F. *et al.* Grounded SAM: Assembling open-world models for diverse visual tasks. *arXiv preprint arXiv:2401.14159*, 2024.

Schneider, S., Greenberg, S., Taylor, G. W., and Kremer, S. C. Three critical factors affecting automated image species recognition performance for camera traps. *Ecology and evolution*, 10(7):3503–3517, 2020.

Sherley, R. B., Burghardt, T., Barham, P. J., Campbell, N., and Cuthill, I. C. Spotting the difference: towards fully-automated population monitoring of African penguins Spheniscus demersus. *Endangered Species Research*, 11:101–111, 2010.

Sherley, R. B., Crawford, R. J. M., Dyer, B. M., Hagen, C., Upfold, L., McInnes, A., and Masotla, M. J. Updated population trajectories and conservation status of the African penguin in South Africa following the 2021 census. Technical Report FISHERIES/2021/JUL/SWG-PEL/46, Department of Forestry, Fisheries and the Environment, 2021.

Srinithi, K., Sushma, S., Aron Nambikai Fernando, J., Srivarshini, S., Kalpana, B., and Karthi, M. Wildlife Recognition in Trap Images: Assessing the Efficiency of YOLO Frameworks. In *2024 International Conference on Integration of Emerging Technologies for the Digital World (ICIETDW)*, pages 1–6. IEEE, 2024.

Tuia, D., Kellenberger, B., Beery, S., Costelloe, B. R., Zuffi, S., Risse, B., Mathis, A., Mathis, M. W., Van Langevelde, F., Burghardt, T. *et al.* Perspectives in machine learning for wildlife conservation. *Nature communications*, 13(1):792, 2022.

Ultralytics. Ultralytics YOLO Documentation. https://docs.ultralytics.com/, 2026. Accessed: 1 October 2026.

Van den Berg, M. S. Pose Estimation for the Critically Endangered African Penguin. Ph.D. thesis, Stellenbosch: Stellenbosch University, 2025.

Wang, C., Song, X., Wang, J., and Yan, X. An improved YOLOv11 algorithm for small object detection in UAV images. *Signal, Image and Video Processing*, 19(6):507, 2025a.

Wang, C.-Y., Liao, H.-Y. M., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., and Yeh, I.-H. CSPNet: A new backbone that can enhance learning capability of CNN. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pages 390–391. 2020.

Wang, L., Ye, L., Chen, X., and Chu, N. MSFN-YOLOv11: A novel multi-scale feature fusion recognition model based on improved YOLOv11 for real-time monitoring of birds in wetland ecosystems. *Animals*, 15(23):3472, 2025b.

Wu, J., Xu, W., He, J., and Lan, M. YOLO for penguin detection and counting based on remote sensing images. *Remote Sensing*, 15(10):2598, 2023.

Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., and Chen, J. DETRs Beat YOLOs on Real-time Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16965–16974. June 2024.

Zunin, V. Intel OpenVINO toolkit for computer vision: Object detection and semantic segmentation. In *2021 International Russian Automation Conference (RusAutoCon)*, pages 847–851. IEEE, 2021.

A

Appendix

This appendix presents additional results referenced in Chapter 4.

A.1 Training Curves

Figures A.1 and A.2 show the training box loss and validation mAP@50:95 of each model during training on the original and grouped splits. In both splits, the training box loss rose briefly during the first few epochs, while the learning rate was warming up, and then fell steadily. It dropped sharply when mosaic augmentation was switched off for the final 10 epochs, as the remaining images were easier to fit. The medium models reached a lower box loss than the nano models, consistent with their higher accuracy. The losses of YOLOv11 and YOLOv26 are not directly comparable, as YOLOv26 trains without Distribution Focal Loss. Validation mAP@50:95 rose quickly over the first 20 epochs and then levelled off, and both nano models were still improving slightly at epoch 100 on the grouped split (Section 3.3).

A.2 Counting by Number of Penguins per Photograph

Figure A.3 shows the counting results of Table 4.3 as error per photograph and error in the total count, for each group of photographs.

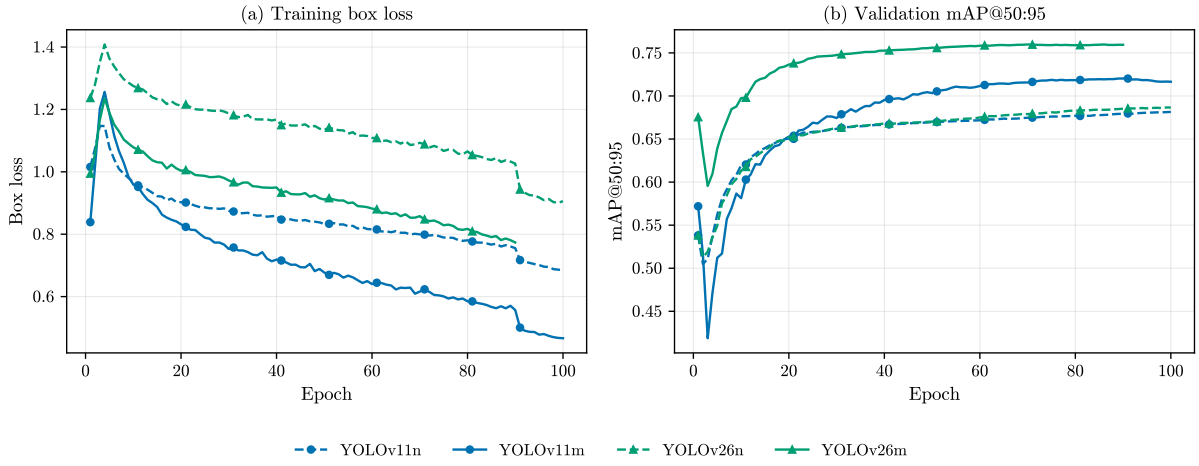


Figure A.1: Training on the original split (Experiment 1.1): training box loss (a) and validation mAP@50:95 (b) per epoch.

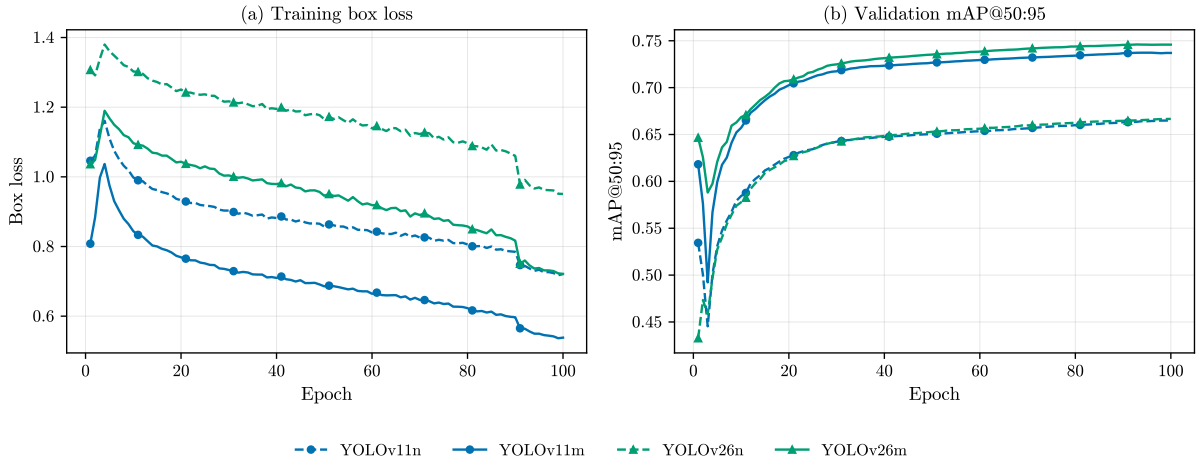


Figure A.2: Training on the grouped split (Experiment 1.2): training box loss (a) and validation mAP@50:95 (b) per epoch.

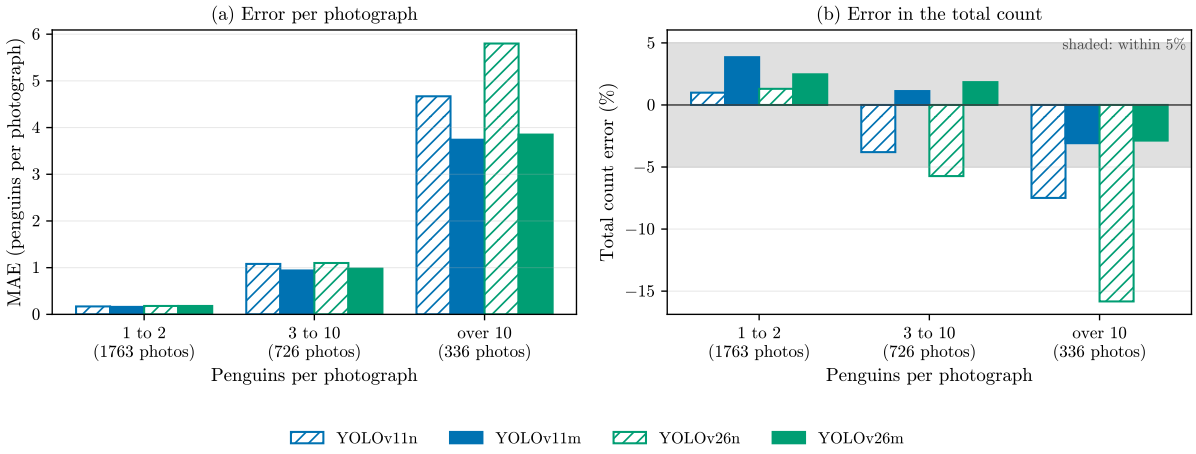


Figure A.3: Counting error of each model by the number of penguins per test photograph, on the grouped split at a confidence threshold of 0.25. The shaded band in (b) marks the 5% target.

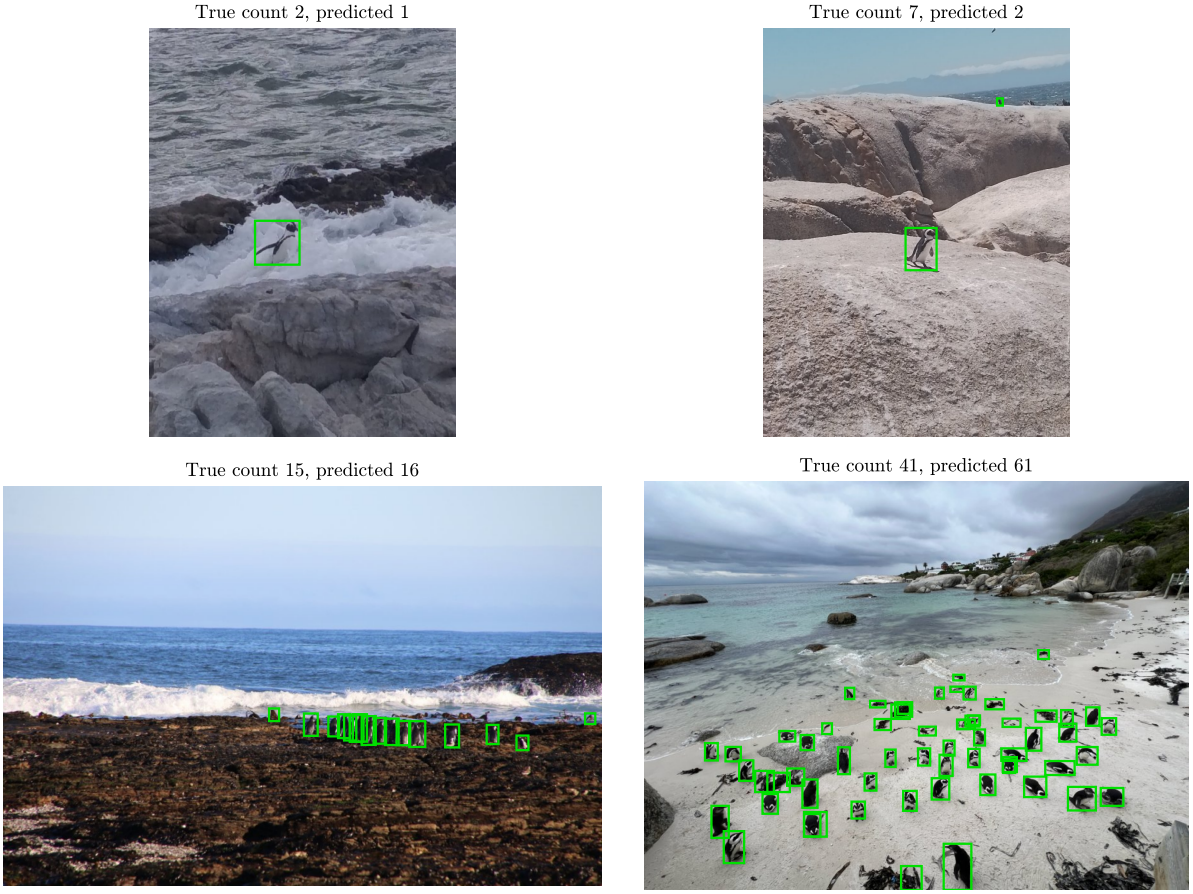


Figure A.4: YOLOv11n detections on four African Penguin test photographs from iNaturalist, at a confidence threshold of 0.25. Each title gives the true and predicted counts.

A.3 Example Detections

A.4 Full FP32 and FP16 Results

Table A.1: OpenVINO models at FP32 and FP16 on the grouped-split test set, timed in the second benchmarking session. "All" is the full test set and "African" its 835 African Penguin photographs.

			All		African			
Model	**Prec.**	**MB**	**mAP@50**	**mAP@50:95**	**mAP@50**	**mAP@50:95**	**ms**	**FPS**
YOLOv11n	FP32	10.7	0.8616	0.6568	0.8974	0.6713	15.32 ± 0.88	65.3
	FP16	5.6	0.8615	0.6567	0.8974	0.6712	15.54 ± 0.96	64.3
YOLOv26n	FP32	10.0	0.8543	0.6572	0.8884	0.6697	14.81 ± 1.07	67.5
	FP16	5.3	0.8542	0.6572	0.8884	0.6697	15.02 ± 1.17	66.6
YOLOv11m	FP32	80.6	0.8996	0.7276	0.9271	0.7434	110.59 ± 2.99	9.0
	FP16	40.6	0.8996	0.7275	0.9271	0.7433	109.41 ± 3.04	9.1
YOLOv26m	FP32	81.9	0.9007	0.7376	0.9308	0.7564	110.81 ± 2.81	9.0
	FP16	41.3	0.9007	0.7375	0.9308	0.7564	110.37 ± 3.05	9.1

A.5 Latency Across Benchmarking Sessions

The OpenVINO FP32 models were timed in two separate runs: first in Experiment 2, and again in Experiment 3, alongside the FP16 models. Table A.2 compares the two runs, which used the same models, test photographs and settings.

Table A.2: OpenVINO FP32 latency per image (± standard deviation) in the two benchmarking runs, on the same 3,383 test photographs. Difference is the change from the Experiment 2 run to the Experiment 3 run, as a percentage of the Experiment 2 run.

Model	**Experiment 2 run (ms)**	**Experiment 3 run (ms)**	**Difference**
YOLOv11n	15.90 ± 0.90	15.32 ± 0.88	3.6%
YOLOv26n	14.77 ± 1.06	14.81 ± 1.07	0.3%
YOLOv11m	111.01 ± 2.67	110.59 ± 2.99	0.4%
YOLOv26m	111.46 ± 2.70	110.81 ± 2.81	0.6%

A.6 Hyperparameter Tuning Trials

Table A.3 lists the ten tuning trials of Experiment 4.2, and Figure A.5 shows how each searched value relates to the validation score. The learning rate alone explained 96% of the variation between trials. Once its trend was removed, neither momentum nor weight decay showed a clear effect.

Table A.3: The ten tuning trials of YOLOv11n on the grouped split (Experiment 4.2), ordered by learning rate. Each trial continued the baseline for 20 epochs. The baseline scored 0.6651.

Learning rate	Momentum	Weight decay	Val mAP@50:95
0.00055	0.862	4.5×10^{-4}	**0.6397**
0.00112	0.910	4.8×10^{-4}	0.6339
0.00151	0.919	4.0×10^{-4}	0.6311
0.00170	0.868	2.3×10^{-4}	0.6303
0.00221	0.971	4.3×10^{-4}	0.6273
0.00229	0.942	8.4×10^{-5}	0.6269
0.00238	0.978	7.3×10^{-5}	0.6238
0.00283	0.975	2.4×10^{-5}	0.6241
0.00311	0.896	1.7×10^{-4}	0.6247
0.00454	0.930	4.4×10^{-5}	0.6214

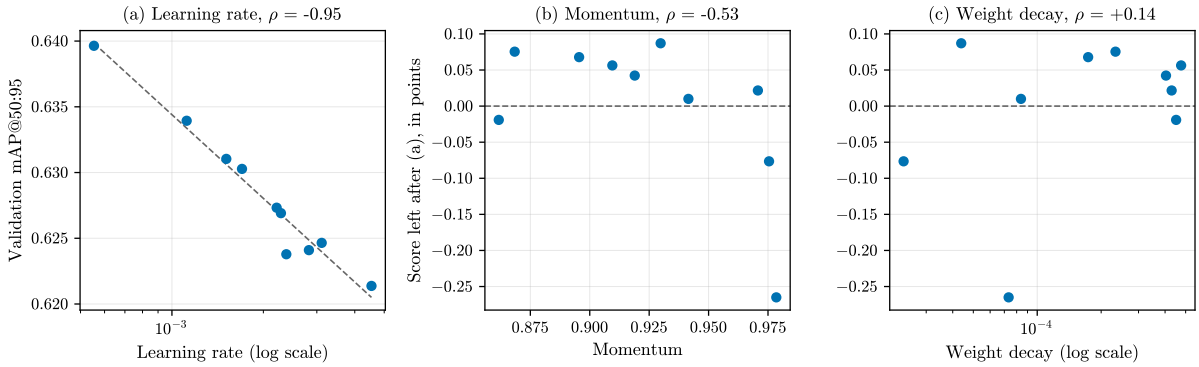


Figure A.5: The ten tuning trials of Experiment 4.2. (a) Validation mAP@50:95 against learning rate, with a fitted trend. (b, c) The score remaining after the learning-rate trend is removed, against momentum and weight decay. ρ is Spearman's rank correlation.

A.7 Scale Augmentation on African Penguins

Table A.4: Experiment 5 detection results for YOLOv11n on the full test set and on its African Penguin photographs.

	All		**African**	
Scale	**mAP@50**	**mAP@50:95**	**mAP@50**	**mAP@50:95**
0.5 (default)	**0.8616**	**0.6568**	**0.8974**	**0.6713**
0.3	0.8596	0.6521	0.8946	0.6653